UNIVERSIDAD POLITÉCNICA DE VICTORIA

IMPLEMENTACIÓN DE FLUENTES DE ESTADO MULTIVALUADOS EN MDP-PROBLOG MEDIANTE DISYUNCIONES ANOTADAS.

T E S I N A

QUE PARA OBTENER EL GRADO DE

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

PRESENTA:

ÁNGEL IVÁN CABRERA ROJAS

DIRECTOR

DR. SAID POLANCO MARTAGÓN

CO-DIRECTOR

DR. HÉCTOR HUGO AVILÉS-ARRIAGA

ORGANISMO RECEPTOR

UNIVERSIDAD POLITÉCNICA DE VICTORIA

CIUDAD VICTORIA, TAMAULIPAS, FEBRERO DE 2026

Ciudad Victoria, Tamaulipas, a **25 de Enero de 2026**

UNIVERSIDAD POLITÉCNICA DE VICTORIA
DR. HÉCTOR HUGO AVILÉS ARRIAGA
PRESENTE

La Universidad Politécnica de Victoria tiene a bien presentar a **CABRERA ROJAS ANGEL IVAN** estudiante del programa académico de **INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN**, con número de matrícula **2230343** y seguro facultativo IMSS número **86-18-04-3681-7;** quién deberá realizar su práctica profesional de **ESTADÍA**, a partir del **07 de Enero de 2026** al **17 de Abril de 2026**, con duración de **600 horas** desarrollando el proyecto **"Implementación de disyunciones anotadas en MDP-ProbLog"** que le fué asignado.

Al concluir se le extenderá la carta de liberación al evaluarlo satisfactoriamente y además le solicitaremos su valiosa opinión respondiendo el formulario que se le enviará por mail, referente al desempeño del practicante, la información que nos proporcione, es de vital importancia para la mejora de los programas académicos que ofrece la Universidad Politécnica de Victoria para la formación de profesionistas altamente especializados.

El practicante deberá cumplir con el Reglamento Interno aplicable al personal en su centro de trabajo.

Sin otro particular.

ATENTAMENTE

OTHÓN CANO GARZA
DIRECTOR DE VINCULACIÓN

C.C.P. SAIDPOLANCOMARTAGÓN
ASESOR INSTITUCIONAL

UNIVERSIDAD POLITÉCNICA DE VICTORIA

Victoria Tamaulipas, a 25 de Enero del 2026
Asunto: Carta de Aceptación

M.A OTHÓN CANO GARZA
DIRECTOR DE VINCULACIÓN
UNIVERSIDAD POLITÉCNICA DE VICTORIA
PRESENTE

Hacemos de su conocimiento que hemos aceptado al estudiante **CABRERA ROJAS ANGEL IVAN** del programa académico de **INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN**, con número de matrícula **2230343** de la Universidad Politécnica de Victoria, para realizar su **ESTADÍA** en nuestra empresa **UNIVERSIDAD POLITÉCNICA DE VICTORIA**, durante el periodo comprendido del día **07 de Enero de 2026** al **17 de Abril de 2026**, el estudiante estará colaborando en el proyecto **"Implementación de disyunciones anotadas en MDP-ProbLog"** con una carga horaria total de **600 horas**

Al concluir satisfactoriamente sus prácticas profesionales, se le entregará al estudiante su carta de liberación debidamente formalizada.

Con el objetivo de colaborar en la mejora de los programas académicos de la Universidad Politécnica de Victoria, estamos de acuerdo en responder a la brevedad posible el formulario de evaluación del desempeño del practicante previo a emitir la liberación de **ESTADÍA**.

Sin otro particular.

ATENTAMENTE
DR. HÉCTOR HUGO AVILÉS ARRIAGA
ASESOR EMPRESARIAL

C.C.P. ARCHIVO

Ing. Juan Camaney de Alba Rojas
PRESENTE

INSERTAR AQUÍ
DOCUMENDO
DE CARTA
DE LIBERACION
DEBIDAMENTE
FIRMADO

CARTA DE ACEPTACIÓN DEL DOCUMENTO PARA SU IMPRESIÓN

Cd. Victoria, Tamaulipas a x

Ángel Iván Cabrera Rojas
PRESENTE

Le comunico que el Programa Académico de Ingeniería en Tecnologías de la Información le ha otorgado la autorización para la impresión de su Tesina de Estadía Práctica cuyo título es:

Implementación de fluentes de estado multivaluados en MDP-ProbLog mediante disyunciones anotadas.

ATENTAMENTE

Dr. Said Polanco Martagón
ASESOR INSTITUCIONAL

c.c.p Director de programa académico

EVALUACIÓN DE ESTADÍA
Rúbrica para evaluación de la presentación y el reporte de estadía

Nombre del alumno: **ÁNGEL IVÁN CABRERA ROJAS** Calificación final: _____

Periodo: **X**

Ponderación	Aspecto a Evaluar	Competente 10	Independiente 9	Básico Avanzado 8	No Competente 5
40	Resultados y Actividades	Estrechamente relacionados al perfil de egreso de su programa académico	Parcialmente relacionados al perfil de egreso de su programa académico	Escasamente relacionados al perfil de egreso de su programa académico	Escasamente relacionados al perfil de egreso de su programa académico
30	Exposición de las actividades de la estadía	Detalladas y sustentadas con respecto a los resultados que se obtuvieron	Detalladas y sustentadas parcialmente con respecto a los resultados que se obtuvieron	Detalladas parcialmente con respecto a los resultados que se obtuvieron	Detalladas escazamente con respecto a los resultados que se obtuvieron
10	Material visual Lenguaje verbal	Uso el lenguaje y la terminología apropiadas; El material visual está organizado, adecuado y suficiente	Uso el lenguaje y la terminología apropiadas El material visual está parcialmente organizado y es suficiente	Uso el lenguaje y la terminología son parcialmente apropiadas; El material visual está parcialmente organizado y es suficiente	Uso el lenguaje y terminología es inapropiado; El material visual no está organizado y es insuficiente
10	Exposición en Idioma Inglés	Pronunciation is clear so language is easily understood (2.5) Uses fluent connected speech, occasionally disrupted by search for correct form of expression (2.5) Uses topic related vocabulary without problems (2.5) Responds to questions using varied and descriptive vocabulary and language structures (2.5)	Pronunciation is understandable, but there are slight errors (2.25) Speech is connected but frequently disrupted by search for correct form of expression (2.25) Uses some topic related vocabulary sufficient to communicate ideas (2.25) Responds to questions using simple but accurate vocabulary and language structures (2.25	Pronunciation is understandable most of the time, marked native accent and many errors (2) Speaks with simple sentences, sometimes not connected, but is understood (2) Uses basic vocabulary to communicate ideas (2) Partly responds to simple questions, with limited vocabulary and language structures (2)	Pronunciation makes language very difficult to understand (1) Uses one-word/two-word utterances (1) Unable to communicate ideas due to lack of vocabulary (1) Uses isolated words or sentence fragments to respond to questions (1)
5	Respuesta a los cuestiona-mientos de los evaluadores	Clara y satisfactoria	Clara y parcialmente satisfactoria	Clara e insuficiente	Confusa e insuficiente
5	Autorización de tesina en tiempo y forma	Presenta en tiempo y forma	Presenta en tiempo y forma con la mayoría de requerimientos solicitados	Presenta en tiempo y con algunas limitantes de los requerimientos solicitados.	Presenta fuera de tiempo y con los mínimos requerimientos solicitados.

Dr. Said Polanco Martagón	X	
ASESOR INSTITUCIONAL	EVALUADOR	EVALUADOR DE INGLÉS

REGISTRO DE EVALUACIÓN DE EXPOSICIÓN DE ESTADÍA

Siendo las X horas del día X, el alumno **Ángel Iván Cabrera Rojas**, del programa académico **Ingeniería en Tecnologías de la Información**, con matricula **2230343**, presentó la exposición de la estadía realizada durante el cuatrimestre **x**, en la **Universidad Politécnica de Victoria**, con el proyecto titulado **Implementación de fluentes de estado multivaluados en MDP-ProbLog mediante disyunciones anotadas.**.

Una vez concluido el proceso de evaluación, y con base a la rúbrica establecida para éste propósito, se determina que la calificación de la estadía es _____.

_____
Dr. Said Polanco Martagón
ASESOR INSTITUCIONAL

_____
X
EVALUADOR

_____

EVALUADOR DE INGLÉS

Agradecimientos

Este trabajo representa el final de días que comienzan temprano en la mañana. De viajes con lo que ahora son cientos de horas, minutos y segundos acumulados. Días de llegar a la universidad y encontrarme con amigos, de volver a casa y pensar sobre qué pasará mañana. De recorrer las mismas calles, escuchar la misma canción, ver a las mismas personas y preguntarme, ¿Qué pasará mañana?.

Aunque muchos de mis días

Aunque la única rutina que tal vez no vuelva a experimenta sea el llegar a la universidad, es cierto que realizar ninguna de ellas se sentirá igual a cómo las he vivido estos últimos tres años. Agradezco profundamente a mis amigos, por ser la

Gracias a todos los que me han acompañado y que en todo sentido, han influido en el desarrollo de mi persona. Gracias a mis amigos, a mis compañeros de clase y a mis profesores, especialmente al Dr. Hector Hugo Avilés-Arriaga, por brindarme la oportunidad de trabajar en este proyecto, con su continuo apoyo y guía.

Pensar en todo

Los agradecimientos más especiales a quien más se lo merecen. Mis padres, Gloria Maribel Rojas García y Jose Ángel Cabrera Rojas y mi hermana Ivanna Amelie Cabrera Rojas.

Resumen

La Programación Lógica Probabilística (PLP) es un paradigma que combina la capacidad de representación de la lógica de primer orden con la teoría de la probabilidad para el manejo de la incertidumbre. Este paradigma permite modelar una amplia variedad de dominios que abarcan situaciones de la vida real en donde existen interacciones complejas entre conjuntos de elementos y distintos niveles de incertidumbre. Dentro de este dominio, el framework MDP-ProbLog utiliza las capacidades del lenguaje ProbLog para la especificación y resolución de Procesos de Decisión de Markov (MDPs) de horizonte infinito utilizando una representación factorizada, lógica y relacional.

Sin embargo, la arquitectura base de este sistema presenta una limitación estructural ya que la representación de los fluentes de estado se restringe exclusivamente a variables de Bernoulli independientes (booleanas). Debido a esto, para describir variables que por naturaleza son multivaluadas, el usuario se ve obligado a utilizar técnicas para descomponerlas en múltiples predicados binarios. En el caso de MDP-ProbLog, esta restricción resulta en un proceso de modelado tedioso, poco práctico y propenso a inconsistencias semánticas.

Se reporta el desarrollo de una extensión al núcleo del framework MDP-ProbLog para solucionar esta problemática. El trabajo abarca la revisión de la arquitectura del código fuente, el planteamiento y diseño de la extensión, la programación, la ejecución de pruebas y su respectiva documentación. Mediante modificaciones a los módulos de gestión de estados y del motor de inferencia, se integró el soporte para fluentes de estado multivaluados utilizando el mecanismo nativo de las Disyunciones Anotadas (Annotated Disjunctions). Como resultado de este desarrollo, el modelado de MDPs que requieren variables con dominios multivaluados se convierte en un proceso semánticamente íntegro, mucho más directo y natural para el usuario.

Palabras clave: Programación Lógica Probabilística, MDP-ProbLog, Disyunciones Anotadas, Fluentes Multivaluados, Espacio de Estados.

Summary

Probabilistic Logic Programming (PLP) is a paradigm that combines the representational power of first-order logic with probability theory to handle uncertainty. This paradigm allows for the modeling of a wide variety of domains encompassing real-world situations where complex interactions exist among sets of heterogeneous elements and varying levels of uncertainty. Within this context, the MDP-ProbLog framework stands out, leveraging the capabilities of the ProbLog language for the specification and resolution of infinite-horizon Markov Decision Processes (MDPs) using a factored, logical, and relational representation.

However, the base architecture of this system presents a structural limitation: the representation of state fluents is restricted exclusively to independent Bernoulli (Boolean) variables. Due to this, to model variables that are naturally multi-valued, the user is forced to use techniques to decompose them into multiple binary predicates. In the case of MDP-ProbLog, this restriction results in a modeling process that is tedious, prone to semantic inconsistencies, and unnatural.

This thesis reports the development of an extension to the core of the MDP-ProbLog framework to address this problem. The work encompasses the review of the source code architecture, the planning and design of the extension, programming, testing, and its respective documentation. Through modifications to the state management modules and the inference engine, support for multi-valued state fluents was integrated using the native mechanism of Annotated Disjunctions. As a result of this development, the modeling of MDPs requiring variables with multi-valued domains becomes a semantically sound, much more direct, and natural process for the user.

Keywords: Probabilistic Logic Programming, MDP-ProbLog, Annotated Disjunctions, Multi-valued Fluents, State Space.

Índice

Agradecimientos vii

Resumen viii

Summary ix

Índice x

1. Introducción 1
- 1.1. Antecedentes . . . 2
- 1.2. Definición del Problema . . . 4
- 1.3. Objetivos . . . 5
  - 1.3.1. Objetivo General . . . 5
  - 1.3.2. Objetivos Específicos . . . 5
- 1.4. Justificación . . . 6
- 1.5. Alcances y Limitaciones . . . 7
  - 1.5.1. Alcances . . . 7
  - 1.5.2. Limitaciones . . . 8

2. Estado del Arte 9
- 2.1. MDPs Factorizados y el Problema de la Representación de Estados . . . 9
- 2.2. Programación Lógica Probabilística para la Resolución de MDPs . . . 10
- 2.3. Disyunciones Anotadas como mecanismo de exclusión mutua . . . 11
- 2.4. Compilación de conocimiento para inferencia eficiente . . . 11
- 2.5. Análisis Comparativo . . . 12
- 2.6. Conclusión del Estado del Arte . . . 13

3. Marco Teórico 14
- 3.1. Grid Estocástico . . . 14
- 3.2. Programación Lógica . . . 15
- 3.3. Programación Lógica Probabilística . . . 16
  - 3.3.1. Semántica de distribución . . . 16
- 3.4. Lenguaje ProbLog . . . 17
- 3.5. Disyunciones Anotadas . . . 18
- 3.6. Conteo de Modelos Ponderados y Compilación de Conocimiento . . . 20
- 3.7. Procesos de Decisión de Markov . . . 21
- 3.8. MDPs Factorizados y Fluentes de Estado . . . 23
- 3.9. El framework MDP-ProbLog . . . 23

4. Diseño e Implementación 25
- 4.1. Análisis de la arquitectura original . . . 25
  - 4.1.1. Componentes del sistema original . . . 25
  - 4.1.2. Flujo de ejecución . . . 27
  - 4.1.3. Limitaciones en la gestión de fluentes . . . 27

4.2. Esquema factorizado del espacio de estados . . . 28
4.2.1. Modelo de datos . . . 28
4.2.2. Codificación *mixed-radix* . . . 29
4.2.3. Ejemplo mixed-radix . . . 30
4.2.4. Operaciones relevantes . . . 30
4.3. Clasificador automático de fluentes . . . 31
4.3.1. Modos de declaración . . . 31
4.3.2. El índice invertido de Disyunciones Anotadas . . . 31
4.3.3. Pipeline de clasificación . . . 32
4.3.4. Inferencia implícita de tipo . . . 32
4.4. Pipeline de preparación del MDP . . . 34
4.4.1. Fases del pipeline . . . 34
4.4.2. Hechos *dummy* y condicionamiento por evidencia . . . 34
4.4.3. Corrección de la compilación doble . . . 36
4.4.4. Declaración de fluentes multivaluados . . . 36
4.5. Adaptación de la Iteración de Valor . . . 37
4.6. Evaluador de Darwiche . . . 39
4.6.1. El costo de la evaluación estándar . . . 39
4.6.2. La idea central: derivadas como respuestas . . . 39
4.6.3. El algoritmo de diferenciación en dos pasadas . . . 40
4.6.4. Implementación y activación . . . 41
4.7. Herramientas auxiliares . . . 42
4.7.1. Exportador de resultados . . . 42
4.7.2. Simulador de trayectorias . . . 43
4.8. Retrocompatibilidad . . . 44

5. Experimentación y Resultados **46**
5.1. Diseño experimental . . . 46
5.1.1. Dominios de evaluación . . . 46
5.1.2. Variables independientes . . . 46
5.1.3. Protocolo de medición . . . 46
5.2. Entorno de evaluación . . . 47
5.3. Corrección funcional . . . 47
5.3.1. Verificación en el dominio Mitchell . . . 47
5.4. Simplificación del modelado . . . 48
5.5. Impacto computacional . . . 50
5.5.1. Tiempos de ejecución . . . 50
5.5.2. Cociente de tiempos entre codificaciones . . . 51
5.5.3. Comparación de configuraciones de inferencia . . . 51
5.6. Dominio Russell . . . 53
5.6.1. Verificación de la función de valor óptima . . . 53
5.7. Discusión . . . 54

6. Conclusiones **55**
6.1. Síntesis del trabajo realizado . . . 55

6.2. Contribuciones 55
6.3. Limitaciones técnicas identificadas 56
6.4. Trabajo futuro 57

Índice de figuras **58**

Índice de cuadros **59**

Índice de algoritmos **60**

Referencias **61**

1. Introducción

Dentro del ámbito de la Inteligencia Artificial, la Programación Lógica Probabilística se ha consolidado como un paradigma expresivo y formalmente fundamentado para la representación del conocimiento bajo incertidumbre, al combinar la capacidad deductiva de la programación lógica con el tratamiento de la probabilidad [1]. Uno de los desarrollos más representativos de este paradigma es *ProbLog*, un lenguaje que extiende Prolog mediante la introducción de hechos probabilísticos y que permite calcular la probabilidad de que una consulta lógica sea verdadera dado un conjunto de evidencias.

Asimismo, en el campo de la toma de decisiones secuenciales, los Procesos de Decisión de Markov (MDPs) se han establecido como el estándar matemático para modelar agentes que actúan en entornos estocásticos. Formalizados a mediados del siglo XX con los trabajos de Richard Bellman sobre programación dinámica [2], los MDPs proporcionan un marco formal para optimizar una función de utilidad a lo largo del tiempo. En su formulación tabular clásica, cada estado del sistema se trata como una entidad indivisible y las transiciones se almacenan en una tabla que lista explícitamente la probabilidad de pasar de cada estado a cada otro estado bajo cada acción posible. A medida que aumenta el número de variables que describen el estado del sistema, el tamaño de dicha tabla crece de manera exponencial, un fenómeno que Bellman denominó la "maldición de la dimensionalidad" [2]. Esta particularidad, hace que tanto la especificación como la resolución de MDPs se vuelvan computacionalmente costosas conforme crece la complejidad del dominio [3].

Integrando la expresividad lógica de ProbLog con la capacidad de decisión de los MDPs, Bueno et al. propusieron el framework **MDP-ProbLog** [4]. Esta herramienta codifica estados, acciones y recompensas dentro de la sintaxis de la programación lógica, permitiendo representar o, de manera relacional y compacta MDPs de horizonte infinito, es decir, problemas de decisión secuencial en los que no hay un punto de terminación predefinido y donde el agente busca una estrategia óptima a largo plazo. En lugar de tratar cada estado como una entidad indivisible, MDP-ProbLog adopta una representación factorizada en la que se descompone el estado global en un conjunto de variables independientes denominadas *fluentes*, de modo que la dinámica del sistema se describe en términos de cómo cambia cada variable individual. Esta representación permite *especificar* el MDP de forma compacta y legible, aunque la resolución del problema mediante el algoritmo de iteración de valor continúa siendo enumerativa y, por tanto, sujeta a la explosión combinatoria del espacio de estados.

1.1. Antecedentes

La idea de utilizar lenguajes declarativos para especificar MDPs no surge con MDP-ProbLog. Boutilier, Dean y Hanks [5] propusieron a finales de la década de 1990 el uso de representaciones factorizadas para abordar la maldición de la dimensionalidad, describiendo las transiciones de estado mediante redes bayesianas dinámicas en lugar de tablas monolíticas. Posteriormente, herramientas como SPUDD [6] explotaron diagramas de decisión algebraicos para resolver MDPs factorizados de manera simbólica.

En el contexto de la Programación Lógica Probabilística, Nitti, Belle y De Raedt [7] demostraron que los lenguajes probabilísticos basados en lógica podían servir como especificación declarativa de problemas de planificación para dominios tanto discretos como continuos. MDP-ProbLog [4] se inscribe en esta línea al proponer una codificación completamente declarativa de MDPs de horizonte infinito dentro de la sintaxis de ProbLog, sustituyendo el muestreo por inferencia exacta mediante Conteo de Modelos Ponderados.

MDP-ProbLog ha sido adoptado como base tecnológica por el equipo de investigación liderado por el Dr. Héctor Hugo Avilés-Arriaga en la Universidad Politécnica de Victoria, el cual lo ha empleado en el desarrollo de sistemas de navegación y toma de decisiones para vehículos autónomos [8]. Sin embargo, al tratarse de un framework que únicamente admite fluentes booleanos, la construcción de los modelos requiere adaptar la descripción del dominio a las limitaciones de la herramienta, en lugar de permitir su representación en términos naturales. Esto en sí mismo es una limitación estructural inherente del framework, ya que todos los fluentes de estado se definen exclusivamente como variables de Bernoulli independientes.

Para ilustrar el problema de manera concreta, considérese un dominio de navegación en cuadrícula, un escenario canónico en la literatura de MDPs [9, 10] en el cual, un agente debe desplazarse desde una posición inicial hasta una celda meta, seleccionando en cada paso una de cuatro acciones de movimiento. La posición del agente compone una partde del estado del sistema y puede describirse mediante dos variables: la fila y la columna que ocupa.

Debido a que cada fluente de estado solo admite dos valores (verdadero o falso), la posición de un agente no puede codificarse directamente como un par de coordenadas, ya que cada coordenada naturalmente toma más de dos valores. Por ello, la representación booleana de MDP-ProbLog resulta insuficiente.

En el caso del grid de 2×3, donde existen 6 celdas posibles, se requieren al menos 3 fluentes booleanos para distinguir 6 estados distintos utilizando únicamente variables de dos valores, ya que 2 variables solo permiten $2^2 = 4$ combinaciones, insuficientes para representar las 6 celdas. Con 3 variables booleanas se obtienen $2^3 = 8$ combinaciones, cantidad suficiente para asignar una combinación única a cada celda.

Como se observa en la Tabla 1, las 8 combinaciones que generan los 3 fluentes booleanos exceden las 6 celdas reales del grid. A lo largo del documento, se denominará *estados espurios* a las combinaciones sobrantes, es decir, configuraciones que el sistema de inferencia considera válidas pero que carecen de correspondencia física en el dominio.

Tabla 1: Codificación binaria de las 6 celdas de un grid de 2×3.

b_1	b_2	b_3	**Celda asignada**
0	0	0	Celda (1,1)
0	0	1	Celda (1,2)
0	1	0	Celda (1,3)
0	1	1	(sin correspondencia)
1	0	0	Celda (2,1)
1	0	1	Celda (2,2)
1	1	0	Celda (2,3)
1	1	1	(sin correspondencia)

La Figura 1 contrasta ambas representaciones del mismo grid: a la izquierda, la descripción mediante las variables de fila (x) y columna (y), que genera exactamente 6 estados; a la derecha, la codificación booleana mediante tres fluentes binarios (b_1, b_2, b_3), que produce 8 combinaciones de las cuales 2 son espurias.

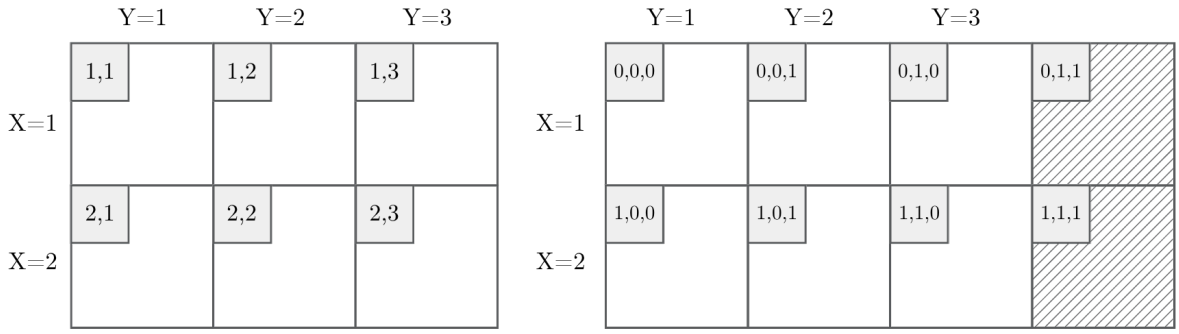

Figura 1: Representación por coordenadas vs. codificación booleana con estados inalcanzables para el grid 2×3.

Este procedimiento de codificación sigue un patrón general. Si se desea representar k estados mutuamente excluyentes se requieren al menos $\lceil \log_2(k) \rceil$ fluentes booleanos, los cuales generan $2^{\lceil \log_2(k) \rceil}$ combinaciones. Siempre que k no sea una potencia exacta de 2, el espacio de estados contendrá combinaciones sin significado claro y que el modelador debe gestionar manualmente. La Tabla 2 ilustra cómo este desajuste crece con el tamaño del dominio.

Tabla 2: Escalamiento de estados espurios en la codificación binaria por tamaño de grid.

Grid	**Celdas** (k)	**Bits necesarios**	**Combinaciones** (2^{bits})	**Espurios**
2×3	6	3	8	2
3×3	9	4	16	7
4×4	16	4	16	0
5×5	25	5	32	7
6×6	36	6	64	28

En un grid de 6×6, por ejemplo, 28 de las 64 combinaciones binarias son espurias, ejemplificando de manera clara las problematicas inherentes a los espacios de estados con variables categóricas mediante la representación que el framework impone actualmente.

1.2. Definición del Problema

Como se ilustró en la sección anterior con el grid de navegación, MDP-ProbLog carece de un mecanismo nativo para representar variables de estado que posean más de dos valores mutuamente excluyentes. Cada fluente se trata como una variable de Bernoulli independiente, lo que obliga a representar cualquier variable multivaluada mediante un conjunto de predicados booleanos cuyas combinaciones codifiquen los valores posibles.

Las consecuencias de esta restricción se extienden a cualquier dominio que requiera variables categóricas. Ejemplos típicos de esto, pueden ser, el estado de un semáforo, el carril que ocupa un vehículo o la fase de un proceso; todas ellas variables que, por su naturaleza, se describen mediante más de dos estados posibles.

En todos estos casos, para representar correctamente este tipo de variables, el usuario debe tener en cuenta estas limitaciones, mientras que el solver puede llegar a ejecutar la iteración de valor sobre un espacio de estados inflado con configuraciones inalcanzables, invirtiendo cómputo en estados que no contribuyen a la política resultante.

Para dimensionar cómo esta limitación afecta el modelado, es útil observar cómo herramientas históricas lidiaron con la misma barrera. La Tabla 3 muestra un ejemplo clásico en la literatura de MDPs factorizados: el dominio de planificación de procesos de manufactura (*Factory*), introducido para evaluar el algoritmo SPUDD [6]. En este escenario se requieren variables categóricas ternarias para modelar con precisión el entorno, tal como la calidad de la pintura de una pieza (buena, deficiente o inexistente).

Tabla 3: Codificación binaria forzada para una variable ternaria en el dominio *Factory*.

Bit 1	**Bit 2**	**Calidad de la pintura**
0	0	Buena (*good*)
0	1	Deficiente (*poor*)
1	0	Inexistente (*false*)
1	1	Estado espurio (*unreachable state*)

Al igual que ocurría en SPUDD, la codificación binaria en MDP-ProbLog introduce inevitablemente estados inalcanzables, lo que provoca que por un lado, wzista un aumento en la complejidad del modelado, debido a que el usuario no puede simplemente declarar los posibles valores de una variable. Esta limitación se traduce en una ineficiencia computacional.

En síntesis, la representación exclusivamente booleana produce modelos que son simultáneamente más costosos de escribir para el humano y menos eficientes de resolver para la máquina, forzando al dominio a adaptarse a las limitaciones de la herramienta.

1.3. Objetivos

1.3.1. Objetivo General

Diseñar e integrar un mecanismo de representación de fluentes de estado multivaluados en el núcleo del framework MDP-ProbLog utilizando Disyunciones Anotadas (Annotated Disjunctions), con el fin de proveer una sintaxis de modelado más natural, asegurar la consistencia en los procesos de decisión estocásticos y evaluar el impacto computacional derivado de esta nueva expresividad.

1.3.2. Objetivos Específicos

- Analizar la arquitectura interna actual de MDP-ProbLog, identificando los módulos críticos encargados de la gestión del espacio de estados y la inferencia probabilística para determinar los puntos de extensión.
- Desarrollar una estructura de datos y adaptar la lógica de instanciación (*grounding*) que permita procesar fluentes multivaluados frente a los fluentes binarios tradicionales.
- Implementar el soporte para Disyunciones Anotadas en la definición de las transiciones de estado, garantizando la exclusión mutua nativa de los valores de una variable.
- Desarrollar las herramientas de inspección que permitan examinar las matrices de transición, las tablas de recompensas y el historial de convergencia de los modelos resueltos.

1.4. Justificación

La extensión propuesta representa un paso necesario para la evolución del framework debido a las limitaciones impuestas por el modelado estrictamente binario. En ausencia de variables categóricas nativas, la arquitectura original transfiere al modelador la tarea de restringir matemáticamente el entorno. Es por ello que usuario debe diseñar y escribir reglas de integridad manuales para evitar que el sistema asigne valores contradictorios a una misma entidad (por ejemplo, evaluar a un agente que ocupa dos posiciones simultáneamente). Esta práctica introduce ruido en el código, oscureciendo la lógica real del proceso de decisión bajo capas de restricciones artificiales.

Se soluciona el problema al trasladar la carga de validación del humano hacia el motor lógico mediante las Disyunciones Anotadas, las cuales son un constructo nativo de la Programación Lógica Probabilística [11] que garantiza la exclusión mutua entre los valores de una variable por construcción semántica. Al delegar esta responsabilidad a la estructura interna de estas disyunciones, el código ProbLog resultante recobra legibilidad y el modelador concentra su esfuerzo en definir la dinámica y las recompensas del entorno, reduciendo la carga cognitiva, acortando los tiempos de desarrollo y eliminando una fuente recurrente de errores de diseño.

Además de las mejoras en la usabilidad, la precisión de esta nueva representación resuelve parte de las deficiencias en el rendimiento computacional. Durante el proceso de resolución original, el algoritmo de iteración de valor desperdicia ciclos de cómputo evaluando configuraciones inalcanzables, invirtiendo recursos en estados que jamás contribuirán a la política resultante. Al garantizar que el espacio de estados contenga únicamente los escenarios que el dominio contempla, el sistema elimina las configuraciones redundantes introducidas por la codificación booleana. Este ahorro computacional es directamente proporcional al número de estados espurios evitados; beneficio que se acumula considerablemente en dominios que utilizan múltiples variables categóricas.

Finalmente, la integración de las Disyunciones Anotadas para modelar fluentes multivaluados no requiere de la modificación de los algoritmos de compilación ni de razonamiento probabilístico. La propuesta extiende la expresividad de la herramienta sin comprometer su arquitectura subyacente ni su retrocompatibilidad con los modelos booleanos existentes.

1.5. Alcances y Limitaciones

1.5.1. Alcances

El proyecto abarca el diseño, la implementación y la validación de una extensión al núcleo del framework MDP-ProbLog que incorpora soporte para fluentes de estado multivaluados. El desarrollo comprende los siguientes elementos:

1. **Representación factorizada del espacio de estados.** El rediseño de las estructuras de datos necesarias para que el espacio de estados admita factores con más de dos valores posibles, preservando la capacidad de iteración y la indexación eficiente sobre el conjunto completo de estados.

2. **Clasificación automática de fluentes.** Se implementó un mecanismo que determina automáticamente si un fluente declarado por el usuario es booleano o multivaluado, a partir del análisis de las Disyunciones Anotadas presentes en el programa. El usuario también puede especificar el tipo de forma explícita.

3. **Integración con el motor de inferencia.** Se adaptó la interfaz entre MDP-ProbLog y el motor de inferencia de ProbLog para soportar la inyección y el manejo de Disyunciones Anotadas como representación nativa de los fluentes multivaluados.

4. **Adaptación del algoritmo de resolución.** Se reestructuró el flujo de evaluación de transiciones y el algoritmo de Iteración de Valor para operar sobre transiciones agrupadas por factor, de modo que cada variable de estado se evalúa de acuerdo con su cardinalidad real.

5. **Herramientas de inspección.** Se desarrollaron utilidades para exportar las matrices de transición, las tablas de recompensas y el historial de convergencia, facilitando la verificación y el análisis de los modelos resueltos.

6. **Retrocompatibilidad.** La extensión preserva el funcionamiento de los modelos booleanos existentes sin requerir modificaciones por parte del usuario.

La validación funcional se realiza sobre dominios basados en grids, específicamente el grid de Mitchell que constituye un problema canónico en la literatura de MDPs con soluciones óptimas conocidas. En cada dominio se contrastan dos formulaciones del mismo problema, una con codificación booleana y otra con fluentes multivaluados, evaluando la simplificación del proceso de modelado, la corrección de la política resultante y el comportamiento del solver en términos de convergencia y tamaño del espacio de estados.

1.5.2. Limitaciones

El desarrollo se restringe a MDPs de horizonte infinito con espacios de estados discretos y completamente observables. Los siguiente puntos delimitan el alcance técnico de la extensión:

1. **Enumeración explícita del espacio de estados.** El alcance de esta extensión se limita a la capa de representación del espacio de estados; la estrategia de resolución enumerativa del framework original permanece sin modificación. El algoritmo de Iteración de Valor continúa recorriendo todos los pares estado-acción en cada iteración, con un costo proporcional al número total de estados por el número de acciones. Dominios con múltiples variables de estado o con variables de cardinalidad alta producen espacios de estados cuya enumeración completa se vuelve computacionalmente costosa.

2. **Capa de representación exclusivamente.** El trabajo no modifica los algoritmos de inferencia exacta ni los compiladores de circuitos lógicos utilizados internamente por ProbLog. El efecto de las Disyunciones Anotadas sobre el tamaño de los circuitos compilados es un resultado emergente del compilador subyacente, no una optimización introducida por esta extensión.

3. **Convención de declaración de fluentes.** Todos los valores posibles de un fluente multivaluado deben declararse bajo el mismo nombre de predicado. Esta convención permite que el sistema identifique automáticamente los grupos mutuamente excluyentes, pero impide representar un fluente cuyos valores se distribuyan entre predicados distintos.

4. **Independencia entre variables de estado.** Las reglas de transición del programa deben respetar independencia entre las variables de estado. El sistema evalúa la probabilidad de transición de cada variable por separado y combina los resultados asumiendo que los factores son estocásticamente independientes. Si las reglas del usuario introducen dependencias entre variables que violan este supuesto, el sistema no lo detecta ni emite advertencias.

5. **Dominios de evaluación.** Las pruebas se ejecutan sobre dominios de grid discretos de dimensión reducida, seleccionados por la disponibilidad de soluciones de referencia en la literatura. Los resultados no se extrapolan a dominios de gran escala ni a entornos físicos en tiempo real.

Una vez delimitado el problema, los objetivos y los alcances de este trabajo, el siguiente capítulo analiza el estado del arte y trabajos que sustentan la propuesta, revisando los enfoques existentes para la representación factorizada de MDPs y el papel de la Programación Lógica Probabilística en la toma de decisiones secuenciales.

2. Estado del Arte

El presente capítulo rastrea la evolución de las tres líneas de investigación cuya intersección define el problema que esta tesina aborda: (1) la representación factorizada de los MDPs y el problema de las variables de estado; (2) el uso de la Programación Lógica Probabilística (PLP) como lenguaje de especificación para la toma de decisiones secuenciales; y (3) los mecanismos de exclusión mutua y compilación de conocimiento para la inferencia eficiente. El recorrido muestra cómo cada uno de ellos ha aportado los elementos necesarios para fundamentar la extensión propuesta.

2.1. MDPs Factorizados y el Problema de la Representación de Estados

A medida que los dominios de planificación crecían en complejidad, quedó claro que ningún método que operara sobre estados individuales podría escalar ya que el tamaño del espacio de estados crece exponencialmente con el número de variables, y los algoritmos clásicos de programación dinámica requieren iterar sobre cada estado [3]. Este crecimiento fue reconocido por la como el obstáculo central que debía resolverse para hacer los MDPs aplicables a problemas reales, y motivó una línea de investigación dedicada a explotar la estructura interna del dominio para eludir la enumeración explícita.

Con este objetivo, Boutilier, Dearden y Goldszmidt propusieron la representación factorizada de MDPs, donde las transiciones se codifican mediante Redes Bayesianas Dinámicas (DBNs) y las funciones de valor se manipulan como árboles de decisión [12]. En esta representación, cada variable de estado posee su propia tabla de probabilidad condicional, y los algoritmos de programación dinámica operan sobre agregaciones de estados en lugar de iterar sobre cada estado individual. Su algoritmo *Structured Policy Iteration* (SPI) demostró que era posible resolver MDPs sin enumerar explícitamente el espacio de estados, siempre que la estructura del problema exhibiera independencias condicionales aprovechables.

En la misma línea, Hoey *et al.* desarrollaron SPUDD, un algoritmo de iteración de valor que reemplaza los árboles de decisión por Diagramas de Decisión Algebraicos (ADDs) [6]. Los ADDs, al ser grafos dirigidos acíclicos, permiten compartir subgrafos isomorfos, logrando reducciones de hasta 30 veces en el número de nodos respecto a las representaciones arbóreas. SPUDD demostró la resolución exacta de MDPs con hasta 63 millones de estados.

Tanto SPI como SPUDD operan exclusivamente sobre variables booleanas debido a la descomposición de Shannon que subyace a ambas estructuras. Cada nodo interno ramifica sobre una variable binaria [6]. Cuando el dominio contiene variables multivaluadas, el modelador debe descomponerlas en múltiples variables binarias. Hoey *et al.* reconocen esta limitación al señalar que una variable ternaria debe expandirse en dos binarias, lo que aumenta el espacio de estados por un factor de 4/3 por variable e introduce estados semánticamente imposibles [6]. Esta binarización forzada constituye precisamente la limitación que se busca corregir en MDP-ProbLog.

En el ámbito de los lenguajes de descripción para planificación, RDDL (*Relational Dynamic Influence Diagram Language*) [13] sí contempla fluentes multivaluados de manera nativa, incluyendo variables enumeradas, enteras y continuas. No obstante, RDDL adopta un DBN estratificado por niveles como representación semántica, donde cada variable de un paso temporal sólo puede depender de variables de niveles anteriores [13]. Esta estructuración acíclica impide modelar dependencias cíclicas entre predicados aterrizados, como las que surgen en dominios con relaciones simétricas o transitivas; MDP-ProbLog resuelve este problema al heredar de Prolog la resolución SLD, capaz de manejar definiciones recursivas sobre el programa aterrizado [4].

2.2. Programación Lógica Probabilística para la Resolución de MDPs

La comunidad de MDPs factorizados desarrolló representaciones cada vez más eficientes del espacio de estados. Años más tarde, la Programación Lógica Probabilística propuso lenguajes relacionales capaces de razonar bajo incertidumbre [14]. La pregunta que eventualmente conectó ambas líneas de investigación fue si los motores de inferencia exacta de ProbLog podían adaptarse para resolver, en lugar de consultas estáticas, problemas de decisión secuencial en los que el agente debe optimizar su comportamiento a lo largo del tiempo.

La extensión de ProbLog hacia la toma de decisiones se materializó primero en DTProbLog [15]. Este sistema añadió al lenguaje la capacidad de modelar problemas de decisión episódicos mediante variables de decisión y atributos de utilidad, permitiendo encontrar la estrategia que maximiza la utilidad esperada. Sin embargo, DTProbLog está diseñado exclusivamente para problemas de un solo paso y no soporta la resolución de MDPs de horizonte infinito, donde es necesario optimizar una política completa a lo largo del tiempo. De esta forma, estableció la prueba de concepto de que ProbLog era extensible hacia la optimización bajo incertidumbre, aunque la resolución de MDPs requería un mecanismo adicional para propagar valor a través de horizontes de tiempo arbitrarios.

Como derivado de este trabajo, Bueno *et al.* propusieron MDP-ProbLog, que extiende la sintaxis de ProbLog con predicados reservados para declarar fluentes de estado, acciones y utilidades, y emplea programas de distribución de transición de dos pasos temporales para representar la dinámica del MDP [4]. El solver implementa iteración de valor sobre el circuito compilado mediante Weighted Model Counting (WMC), demostrando formalmente la equivalencia entre ejecutar el programa probabilístico y realizar un backup de Bellman. Esta equivalencia garantiza que la evaluación sobre el circuito produce exactamente las probabilidades de transición requeridas, permitiendo que el solver converja a políticas óptimas sin enumerar el espacio de estados. Una ventaja distintiva frente a lenguajes como RDDL es su capacidad para representar dependencias cíclicas entre predicados aterrizados gracias a la resolución SLD de Prolog [4].

2.3. Disyunciones Anotadas como mecanismo de exclusión mutua

En paralelo al desarrollo de los MDPs factorizados y de MDP-ProbLog, una línea independiente de investigación dentro de la Programación Lógica Probabilística se ocupaba de un problema distinto: cómo representar eventos con múltiples resultados mutuamente excluyentes sin imponer restricciones externas al programa. Vennekens, Verbaeten y Bruynooghe respondieron a este problema con los *Logic Programs with Annotated Disjunctions* (LPADs) [11], demostrando que es posible codificar la exclusión mutua de manera nativa en la cabeza de las reglas lógicas. Esta propiedad elimina la necesidad de restricciones manuales y evita la generación de estados semánticamente imposibles, dos problemas que los enfoques booleanos independientes trasladan al modelador.

En trabajo posterior, Vennekens, Denecker y Bruynooghe formalizaron una interpretación causal de las ADs mediante *CP-Logic* [16], demostrando que el cuerpo de cada regla representa una causa y los disyuntos en la cabeza sus posibles efectos, cada uno con una probabilidad de ocurrencia. Este alineamiento es directo con la dinámica estocástica de un MDP: dado un estado y una acción, el sistema transita hacia exactamente uno de varios estados siguientes posibles. Los autores demostraron además la equivalencia expresiva entre los LPADs y el Independent Choice Logic de Poole [17], consolidando a las ADs como el mecanismo más natural para representar efectos indeterminados en programas lógicos probabilísticos.

2.4. Compilación de conocimiento para inferencia eficiente

Para que la inferencia probabilística sobre programas con átomos de decisión sea computacionalmente tratable, sistemas como ProbLog emplean un flujo de compilación de conocimiento. El sistema transforma el programa lógico aterrizado en una fórmula proposicional que, a su vez, se compila en un circuito lógico. Sobre este circuito se ejecuta el Conteo de Modelos Ponderados (WMC). Darwiche y Marquis establecieron la taxonomía formal de los lenguajes de compilación e identificaron que estructuras como las d-DNNFs (*Deterministic Decomposable Negation Normal Forms*) permiten realizar inferencia exacta en tiempo lineal respecto al tamaño del circuito compilado [18]. Posteriormente, Darwiche introdujo los Diagramas de Decisión Sentencial (SDDs), una representación canónica que generaliza los clásicos BDDs y resulta estrictamente más sucinta [19].

Un avance directamente relevante para este trabajo es el algoritmo de diferenciación de circuitos propuesto por Darwiche [20], que permite calcular todas las derivadas parciales de un circuito d-DNNF en dos pasadas lineales sobre el mismo. Esta técnica amortiza el costo de inferencia cuando se requieren múltiples evaluaciones simultáneas, como ocurre durante la iteración de valor en MDP-ProbLog, donde cada par estado-acción demanda calcular las probabilidades de transición de todos los fluentes del siguiente estado. ProbLog2 integra estas estructuras como backends de compilación, ofreciendo flexibilidad para seleccionar entre d-DNNFs y SDDs según las compensaciones de espacio y tiempo del dominio [21]. Cualquier extensión a la capa de representación, como la integración de fluentes multivaluados propuesta, debe preservar la compatibilidad con este flujo de compilación para mantener las garantías de inferencia exacta.

2.5. Análisis Comparativo

La Tabla 4 compara los ocho enfoques revisados en este capítulo, agrupados por familia metodológica. Para cada uno se evalúan cuatro dimensiones: el tipo de variables de estado que soporta, la exactitud de la inferencia, la capacidad relacional del lenguaje, y el mecanismo de exclusión mutua.

Tabla 4: Comparativa de los enfoques del estado del arte.

Trabajo	**Variables de estado**	**Inferencia**	**Expresividad relacional**	**Exclusión mutua**
MDPs factorizados proposicionales				
Boutilier *et al.* [12]	Booleanas	Exacta	Proposicional	Manual
Boutilier, Dean y Hanks [5]	Booleanas	Exacta	Proposicional	Manual
Hoey *et al.* [6]	Booleanas	Exacta	Proposicional	Manual
Lenguajes de planificación				
Sanner [13]	Multivaluadas / continuas	Aprox.	Relacional	Nativa por tipo
Programación Lógica Probabilística				
De Raedt y Kimmig [14]	Booleanas	Exacta	Relacional	No aplica
Fierens *et al.* [22]	Booleanas	Exacta	Relacional	No aplica
Exclusión mutua y semántica causal				
Vennekens *et al.* [16]	Multivaluadas	Exacta	Relacional	Nativa
MDP-ProbLog				
Bueno *et al.* [4]	Booleanas	Exacta	Relacional	Manual
Propuesta	**Multivaluadas**	**Exacta**	**Relacional**	**Nativa**

Como se observa, la propuesta reúne simultáneamente las cuatro capacidades: variables de estado multivaluadas, inferencia exacta, expresividad relacional y exclusión mutua nativa.

2.6. Conclusión del Estado del Arte

Los MDPs factorizados demostraron que la planificación a escala es viable cuando los algoritmos explotan las independencias del dominio [12, 6]. Posteriormente, MDP-ProbLog combinó la expresividad relacional con la inferencia exacta para resolver MDPs de horizonte infinito [4]. Por su parte, los LPADs y CP-Logic formalizaron la exclusión mutua nativa como una propiedad semántica de los programas lógicos, la cual se alinea perfectamente con la dinámica de las transiciones estocásticas [11]. Finalmente, la compilación de conocimiento proporcionó las estructuras algorítmicas sobre las que este mecanismo evalúa probabilidades en tiempo lineal [18, 20].

La revisión de la literatura evidencia que la programación lógica ofrece herramientas probadas para el modelado multivaluado mediante Disyunciones Anotadas que MDP-ProbLog desaprovecha y cuya corrección conlleva una intervención en la arquitectura del sistema.

Dado que el espacio de estados deja de representarse como una estructura de factores exclusivamente binarios, el motor de resolución necesita adoptar un esquema que permita a cada variable operar bajo su cardinalidad original. Asimismo, el algoritmo de Iteración de Valor debe contemplar el cálculo del valor esperado como una ramificación (n-aria) para así evaluar correctamente las transiciones dictadas por las Disyunciones Anotadas.

Sobre esta base, se diseña e implementa la extensión descrita. El siguiente capítulo establece los fundamentos teóricos detallados que sustentan esta integración, comenzando por las bases de la programación lógica y culminando en las propiedades de los circuitos compilados que hacen computacionalmente posible la toma de decisiones.

3. Marco Teórico

El capítulo anterior estableció que MDP-ProbLog integra la programación lógica probabilística con la resolución de MDPs factorizados. Este capítulo presenta los fundamentos formales para comprender el *framework* y la extensión propuesta. Los conceptos se organizan desde los fundamentos de la lógica hasta el planeamiento bajo incertidumbre.

La Sección 3.2 establece las nociones básicas de programación lógica. Sobre esa base, las Secciones 3.3 y 3.4 introducen la incertidumbre mediante la semántica de distribución y el lenguaje ProbLog. La Sección 3.5 presenta las Disyunciones Anotadas, que incorporan la exclusión mutua como propiedad nativa del formalismo. La Sección 3.6 describe el Conteo de Modelos Ponderados y la compilación de conocimiento. Estos mecanismos transforman la inferencia probabilística en un cálculo tratable.

Las Secciones 3.7 y 3.8 formalizan los Procesos de Decisión de Markov y su versión factorizada. Esta formalización hace visible el costo de representar variables categóricas con fluentes booleanos. Sobre esa base, la Sección 3.9 presenta el *framework* MDP-ProbLog. El Capítulo 4 describe la extensión de su arquitectura para soportar fluentes multivaluados.

3.1. Grid Estocástico

El dominio de referencia es un grid estocástico de 2×3 adaptado de Russell y Norvig [23]. Las celdas se identifican mediante pares (r, c) con $r \in \{1, 2\}$ y $c \in \{1, 2, 3\}$. El agente parte de la celda $(1, 1)$ y debe alcanzar la meta en $(2, 3)$, donde recibe una recompensa de $+10$.

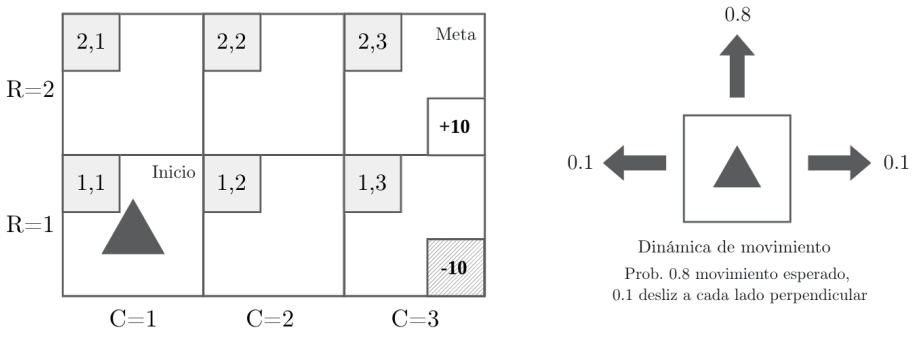

Figura 2: Grid de navegación 2×3 con transiciones estocásticas.

La Figura 2 muestra las posiciones del agente y la meta en el grid. El agente dispone de cuatro acciones: norte, sur, este y oeste. Al ejecutar una acción, avanza en la dirección deseada con probabilidad 0.8 y se desliza hacia cada dirección perpendicular con probabilidad 0.1. Si la celda de destino queda fuera del tablero, el agente permanece en su posición actual.

3.2. Programación Lógica

MDP-ProbLog representa sus estados y transiciones mediante el lenguaje de la programación lógica. Esta sección establece la sintaxis del lenguaje, el proceso de resolución y la noción de instanciación que fundamentan la extensión probabilística de la sección siguiente.

Un programa lógico se construye a partir de hechos, reglas y consultas [23, 1]. Un hecho es una cláusula sin cuerpo que establece la verdad de un átomo. Un átomo es un predicado aplicado a sus argumentos. En el grid, el hecho `posicion(1,1).` afirma que el agente ocupa la celda $(1, 1)$. Una regla establece una implicación entre una cabeza y un cuerpo separados por el operador `:-`. La cabeza es verdadera solo cuando todos los literales del cuerpo son verdaderos.

La regla `meta_alcanzada :- posicion(2,3).` expresa que la condición de éxito se cumple si el agente ocupa la celda $(2, 3)$. Una consulta pregunta si un átomo es derivable a partir de un programa lógico definido. Si existe una prueba, el sistema devuelve la sustitución que lo hace verdadero, de lo contrario, falla.

Un programa lógico definido es un conjunto finito de cláusulas de Horn en el que cada cláusula tiene exactamente un átomo en la cabeza [23]. Las cláusulas pueden incluir variables escritas con mayúscula inicial. Estas se cuantifican universalmente sobre el dominio de objetos. La regla `posicion(R,C) :- celda(R,C).` expresa que (R, C) es una posición válida para todo par que satisfaga `celda`. Esta capacidad reduce el número de cláusulas necesarias para describir espacios de estados extensos.

Prolog responde consultas mediante resolución SLD [1], un proceso descendente sobre la lista de literales pendientes. Para cada literal, el sistema busca una cláusula del programa cuya cabeza pueda unificarse con él. La unificación consiste en encontrar una sustitución que iguale dos términos. Cuando la unificación tiene éxito, se aplica la sustitución y los literales unificados se reemplazan por el cuerpo de la cláusula seleccionada. El proceso concluye con éxito si la lista queda vacía y falla si ninguna cláusula unifica con el literal en curso.

Los átomos base derivables por resolución SLD forman el *modelo mínimo de Herbrand* del programa. Este modelo es el conjunto más pequeño de átomos base que satisface todas las cláusulas del programa [23]. Por ejemplo, para el programa formado por el hecho `posicion(2,3).` y la regla `meta_alcanzada :- posicion(2,3).`, el modelo mínimo es {`posicion(2,3)`, `meta_alcanzada`}. Si el único hecho del programa fuera `posicion(1,1).`, la regla no se activaría y el modelo mínimo sería {`posicion(1,1)`}.

Cuando un programa contiene variables, la instanciación reemplaza cada variable por una constante del dominio [1]. Este proceso, denominado *grounding*, produce un programa aterrizado que contiene únicamente átomos base. La instanciación de `celda(R,C) :- posicion(R,C).`, con $R \in \{1, 2\}$ y $C \in \{1, 2, 3\}$, produce seis cláusulas base, una por cada celda del dominio. El espacio de estados del planificador se construye a partir de estos átomos base, aspecto que se retoma en la Sección 3.9. La sección siguiente introduce la incertidumbre mediante hechos probabilísticos.

3.3. Programación Lógica Probabilística

La Programación Lógica Probabilística (PLP) extiende el lenguaje de la sección anterior con una noción formal de incertidumbre. En MDP-ProbLog, la PLP sustenta el cálculo de toda probabilidad de transición entre estados. Esta sección introduce la semántica de distribución, que formaliza cómo un programa lógico induce una distribución de probabilidad sobre sus mundos posibles.

3.3.1. Semántica de distribución

Sato observó que asignar una probabilidad independiente a cada hecho de un programa lógico hace que las reglas propaguen esa incertidumbre [24]. El resultado es una distribución sobre todos los modelos mínimos posibles. Esta idea, denominada *semántica de distribución*, es el fundamento de los lenguajes de PLP modernos, incluido ProbLog.

Formalmente, un programa se modela como una tupla $DB = F \cup R$, donde $F = \{f_1, \ldots, f_k\}$ es el conjunto de hechos probabilísticos, cada uno con una probabilidad $\theta_i \in [0, 1]$, y R es el conjunto de reglas determinísticas [24]. Los hechos en F son estocásticamente independientes entre sí. Una *realización total* L es una asignación de valores de verdad a todos los hechos de F. Con k hechos independientes y dos valores posibles cada uno, el espacio de realizaciones tiene 2^k elementos.

La probabilidad de cada realización L es el producto de las probabilidades de los hechos verdaderos en L por los complementos de los excluidos:

$$P(L \mid DB) = \prod_{f_i \in L} \theta_i \prod_{f_i \in F \setminus L} (1 - \theta_i) \tag{1}$$

Cada realización L induce un programa lógico definido $L \cup R$ cuyo modelo mínimo de Herbrand es único, denotado $M_{DB}(L)$.

La distribución sobre las realizaciones induce una distribución equivalente sobre los modelos mínimos. Cuando F es finito, Sato demostró que toda la masa de probabilidad se concentra en estos modelos derivados [24]. La probabilidad de éxito de una consulta q es la suma de las probabilidades de todas las realizaciones cuyo modelo mínimo satisface q:

$$P(q \mid DB) = \sum_{L : M_{DB}(L) \models q} P(L \mid DB) \tag{2}$$

Considérese al agente en $(1, 1)$ ejecutando la acción `norte`. La dinámica asigna probabilidad 0.8 al avance hacia $(2, 1)$; el 0.2 restante corresponde a desplazamientos ortogonales. Las Disyunciones Anotadas, que se formalizan en la Sección 3.5, son la herramienta adecuada para modelar estos resultados mutuamente excluyentes. Como punto de partida, el programa agrupa los desvíos en un único hecho de fallo. El Listado 1 codifica este modelo inicial.

```
% Probabilidad agrupada de cualquier desvio ortogonal
0.2::fallo_norte.

% Consecuencias de ir hacia el norte estando en la pos(1,1)
pos(2,1) :- \+fallo_norte.
pos(1,1) :- fallo_norte.

% Consulta
query(pos(2,1)).
```

Código 1: Programa probabilístico. Se agrupan los desvíos en un único fallo.

Este programa define dos mundos posibles. La Tabla 5 detalla cada realización con su probabilidad y su modelo mínimo de Herbrand.

Tabla 5: Mundos posibles inducidos por la semántica de distribución sobre el Código 1.

`fallo_norte`	P(mundo)	`pos(2,1)`	Modelo mínimo
falso	0.8	verdadero	{`pos(2,1)`}
verdadero	0.2	falso	{`fallo_norte`, `pos(1,1)`}

La probabilidad de éxito es $P(\texttt{pos(2,1)}) = 0.8$: la suma de las probabilidades de los mundos donde la consulta es verdadera. La incertidumbre del hecho probabilístico se transmite, a través de las reglas, hasta la consulta.

La Ecuación 2 suma sobre 2^k realizaciones, con k el número de hechos probabilísticos. Esta suma es #P-difícil en el caso general [22]. La compilación de conocimiento, descrita en la Sección 3.6, convierte esta suma en una operación tratable.

La semántica de distribución requiere un lenguaje que la haga operacional. La Sección 3.4 presenta ProbLog, que implementa esta semántica.

3.4. Lenguaje ProbLog

ProbLog [1, 14] implementa la semántica de distribución con una extensión sintáctica mínima sobre Prolog. La extensión consiste en el *hecho probabilístico*, denotado `p::f.`, que declara que el átomo `f` es verdadero con probabilidad p con independencia de los demás hechos. Los átomos, las reglas, las consultas y la resolución SLD conservan el significado establecido en la Sección 3.2.

El motor de inferencia de ProbLog transforma un programa en una respuesta probabilística en cuatro etapas [22]. En la primera, cada variable se sustituye por una constante del dominio. En la segunda, la fórmula resultante se traduce a una fórmula booleana ponderada. En la tercera, esa fórmula se compila a un circuito lógico. En la cuarta, el circuito se evalúa y produce la probabilidad de la consulta. La Sección 3.6 detalla las dos últimas etapas.

El Código 2 refina el modelo anterior con la sintaxis ProbLog. Cada resultado de la transición norte desde $(1, 1)$ se declara como un hecho probabilístico independiente. Las reglas asociadas derivan la celda destino según cuál hecho sea verdadero.

```
% Hechos probabilisticos independientes
0.8::exito_norte.
0.1::desliz_oeste.
0.1::desliz_este.

% Reglas de transicion
pos(2,1) :- exito_norte.
pos(1,1) :- desliz_oeste. % Choca y permanece en su lugar
pos(1,2) :- desliz_este.
```

Código 2: Transición para la acción norte desde $(1, 1)$ con hechos probabilísticos independientes.

Los tres hechos `exito_norte`, `desliz_oeste` y `desliz_este` se declaran estocásticamente independientes. La dinámica del grid exige, sin embargo, que estos eventos sean mutuamente excluyentes. En cada transición, el agente ocupa una única celda destino. La semántica de distribución genera $2^3 = 8$ mundos posibles cuando los tres hechos se tratan como independientes. Uno de esos mundos hace verdaderos los tres hechos con probabilidad $0.8 \times 0.1 \times 0.1 = 0.008$ y produce un modelo mínimo donde el agente ocupa tres celdas a la vez.

Descartar estos mundos imposibles exige restricciones de integridad adicionales. Esas restricciones amplían el espacio de evaluación y oscurecen la descripción del modelo. La Sección 3.5 presenta las Disyunciones Anotadas, que eliminan este problema por construcción semántica.

3.5. Disyunciones Anotadas

Las Disyunciones Anotadas (*Annotated Disjunctions*, ADs), introducidas por Vennekens, Verbaeten y Bruynooghe [11], incorporan la exclusión mutua directamente en la sintaxis de la cláusula. Esta propiedad elimina por construcción los mundos espurios que producen los hechos probabilísticos independientes.

Una Disyunción Anotada es una cláusula de la forma

$$(h_1 : p_1) \vee (h_2 : p_2) \vee \cdots \vee (h_n : p_n) \leftarrow b_1, \ldots, b_m \tag{3}$$

donde $h_1, \ldots, h_n$ son átomos, $p_1, \ldots, p_n$ son probabilidades con $\sum_{i=1}^{n} p_i \leq 1$, y $b_1, \ldots, b_m$ es el cuerpo de la cláusula [11]. Cuando la suma es estrictamente menor a uno, existe una probabilidad residual $1 - \sum_i p_i$ de que ningún head sea derivado. En transiciones de estado de un MDP la suma es igual a uno. Esta condición garantiza que cada instancia de la cláusula produzca un único resultado.

La semántica de las ADs descansa en el concepto de selección. Una selección σ asigna a cada cláusula AD del programa un único head. La instancia P_σ es el programa lógico definido que sustituye cada cláusula AD por el hecho $h_{\sigma(r)}$ de la cláusula r. La probabilidad conjunta de la selección σ, denotada C_σ, es [11]

$$C_\sigma = \prod_r p_{\sigma(r)} \tag{4}$$

donde el producto recorre todas las cláusulas AD del programa. La distribución sobre interpretaciones es la suma de C_σ sobre todas las selecciones cuya instancia P_σ deriva la interpretación objetivo.

El Código 3 reescribe como Disyunción Anotada la transición norte de la Sección 3.4. Esta cláusula genera tres selecciones. La Tabla 6 recoge la distribución resultante.

```
0.8::pos(2,1) ; 0.1::pos(1,1) ; 0.1::pos(1,2).
```

Código 3: Transición norte desde $(1, 1)$ representada como Disyunción Anotada.

Tabla 6: Distribución generada por la AD del Código 3. Cada selección corresponde a un único destino posible.

Selección σ	C_σ	Modelo mínimo
$\sigma \rightarrow$ `pos(2,1)`	0.8	{`pos(2,1)`}
$\sigma \rightarrow$ `pos(1,1)`	0.1	{`pos(1,1)`}
$\sigma \rightarrow$ `pos(1,2)`	0.1	{`pos(1,2)`}

En contraste con los ocho mundos de la Tabla 5, la AD produce tres mundos, uno por destino posible. Ningún mundo combina dos posiciones simultáneas. La selección, por construcción, asigna un único head verdadero en cada mundo.

Los Programas Lógicos con Disyunciones Anotadas (LPADs) son expresivamente equivalentes a la Lógica de Elección Independiente (ICL) de Poole [17] y a CP-Logic [16]. Los tres formalismos definen la misma clase de distribuciones sobre interpretaciones de Herbrand. Sus diferencias residen en la notación empleada y en la interpretación causal que se asigna a las reglas.

ProbLog representa internamente cada AD como un grupo de nodos `choice` en la ClauseDB [22]. Cada nodo `choice` almacena el identificador del grupo y el átomo head asociado. La Sección 3.6 describe cómo esta representación hace tratable la evaluación de probabilidades.

3.6. Conteo de Modelos Ponderados y Compilación de Conocimiento

La selección produce una distribución exacta sobre las interpretaciones, pero calcular la probabilidad de una consulta q sigue exigiendo sumar sobre todas las selecciones compatibles. Esta suma tiene la misma complejidad #P-Hard que la Ecuación 2. El Conteo de Modelos Ponderados (*Weighted Model Counting*, WMC) convierte esa suma en una evaluación aritmética sobre una fórmula compacta del programa.

WMC se aplica sobre la fórmula booleana ponderada que resulta de traducir el programa [22]. Esta traducción define una función de pesos w sobre los literales. A cada átomo probabilístico con probabilidad p se le asigna $w(l) = p$ al literal positivo y $w(\neg l) = 1 - p$ al negativo. Los átomos derivados reciben peso 1 en ambos literales. El peso de una interpretación ω es el producto de los pesos de sus literales:

$$W(\omega, w) = \prod_{l \in \omega} w(l) \tag{5}$$

El WMC de la fórmula φ bajo w es la suma de los pesos de todas sus interpretaciones satisfacibles:

$$\mathrm{WMC}(\varphi, w) = \sum_{\omega \models \varphi} W(\omega, w) \tag{6}$$

Fierens et al. demuestran que la probabilidad de evidencia de un programa ProbLog es igual al WMC de su fórmula ponderada [22]. Las Ecuaciones 6 y 1 revelan que $W(\omega, w)$ coincide con $P(L \mid DB)$. Esta equivalencia transfiere el problema de inferencia probabilística al dominio aritmético de las fórmulas booleanas ponderadas.

El WMC de una fórmula arbitraria sigue siendo #P-Hard. La compilación de conocimiento convierte la fórmula en una estructura que admite WMC en tiempo lineal [18]. Esa estructura es la Forma Normal de Negación Determinista y Descomponible (*d-DNNF*). Una Forma Normal de Negación (NNF) es un grafo dirigido acíclico cuyos nodos internos son conjunciones o disyunciones y cuyos nodos hoja son literales. Una NNF es descomponible si los hijos de cada nodo conjuntivo no comparten átomos entre sí. Una NNF es determinista si los hijos de cada nodo disyuntivo son lógicamente inconsistentes entre sí [18]. Una d-DNNF satisface ambas propiedades.

La descomponibilidad hace que el WMC de cada nodo conjuntivo sea el producto de los WMC de sus hijos. El determinismo hace que el WMC de cada nodo disyuntivo sea la suma de los WMC de sus hijos. La propiedad de suavidad exige que todos los hijos de un nodo disyuntivo mencionen el mismo conjunto de átomos. Las tres propiedades juntas garantizan que el recorrido ascendente sobre el circuito calcule WMC en $O(|C|)$, con $|C|$ el número de

nodos [22, 18]. El circuito aritmético resultante se compila una sola vez por programa; las consultas posteriores no requieren recompilación.

Darwiche demostró que todas las derivadas parciales $\partial\,\mathrm{WMC}(\varphi, w)/\partial w(l)$ para cada literal l del circuito pueden calcularse simultáneamente en dos recorridos lineales [20]. El primero, ascendente, calcula el WMC y los valores intermedios de cada nodo. El segundo, descendente, propaga las contribuciones hacia los nodos hijos. La derivada $\partial\,\mathrm{WMC}/\partial w(l)$ es proporcional a la probabilidad marginal de l bajo la distribución del programa. Todas las probabilidades marginales se obtienen en $O(|C|)$, sin necesidad de evaluaciones independientes del circuito.

ProbLog2 ofrece los Diagramas de Decisión Sentencial (SDDs), introducidos por Darwiche [19], como alternativa al compilador d-DNNF. Los SDDs son un subconjunto estricto de los d-DNNFs que impone propiedades adicionales sobre la descomposición del espacio de variables. Los SDDs soportan además combinación booleana en tiempo polinomial. Esta propiedad facilita la construcción incremental de circuitos cuando se añaden cláusulas al programa.

Durante la Iteración de Valor de MDP-ProbLog, cada respaldo de Bellman requiere evaluar $P(s' \mid s, a)$ para todos los posibles estados siguientes s' [4]. Estas evaluaciones corresponden a múltiples consultas WMC sobre el mismo circuito compilado. El algoritmo de diferenciación de Darwiche obtiene todas estas probabilidades en dos recorridos del circuito, independientemente del número de estados siguientes. La Sección 3.7 formaliza los Procesos de Decisión de Markov. MDP-ProbLog emplea este marco para calcular políticas óptimas.

3.7. Procesos de Decisión de Markov

Los Procesos de Decisión de Markov (MDPs) formalizan la toma de decisiones secuencial bajo incertidumbre. Un agente observa el estado del entorno y selecciona una acción. La acción produce una recompensa y el entorno transita al siguiente estado según una distribución estocástica. El objetivo del agente es maximizar la suma de recompensas futuras descontadas. MDP-ProbLog adopta este modelo para representar el entorno del agente y derivar su política óptima.

Un MDP de horizonte infinito con factor de descuento es una tupla (S, A, T, R, γ), donde S es un conjunto finito de estados, A es un conjunto finito de acciones, $T : S \times A \times S \to [0, 1]$ es la función de transición estocástica con $\sum_{s' \in S} T(s, a, s') = 1$ para todo par (s, a), $R : S \times A \to \mathbb{R}$ es la función de recompensa inmediata y $\gamma \in [0, 1)$ es el factor de descuento [3]. El horizonte infinito con descuento optimiza sobre un futuro indefinido; las recompensas lejanas pesan menos que las inmediatas. La función de valor óptima $V^* : S \to \mathbb{R}$ satisface la ecuación de optimalidad de Bellman [2]:

$$V^*(s) = \max_{a \in A} \left[R(s, a) + \gamma \sum_{s' \in S} T(s, a, s') V^*(s') \right] \tag{7}$$

Esta ecuación establece que el valor óptimo de un estado es la recompensa inmediata máxima más el retorno descontado esperado del estado siguiente. La política óptima $\pi^* : S \to A$ elige en cada estado la acción que maximiza el lado derecho de la Ecuación 7. En el grid de la Sección 3.1, la acción óptima en cada celda depende tanto de la recompensa en la meta como de la distancia restante bajo el factor de descuento γ.

La Iteración de Valor (IV) calcula V^* iterativamente [3]. El algoritmo inicializa $V_0(s) = 0$ para todo s y aplica el operador de Bellman en cada paso:

$$V_{k+1}(s) = \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s,a,s') V_k(s') \right] \tag{8}$$

El algoritmo converge cuando el residuo máximo $\max_{s \in S} |V_{k+1}(s) - V_k(s)|$ cae por debajo del umbral $\varepsilon(1-\gamma)/\gamma$ [3]. La Figura 3 presenta la política π^* y los valores V^* del grid, calculados con $\gamma = 0.9$ y $\varepsilon = 0.1$. Los valores son mayores en las celdas próximas a la meta en $(2,3)$ y decrecen con la distancia. Este patrón refleja el efecto acumulado de γ.

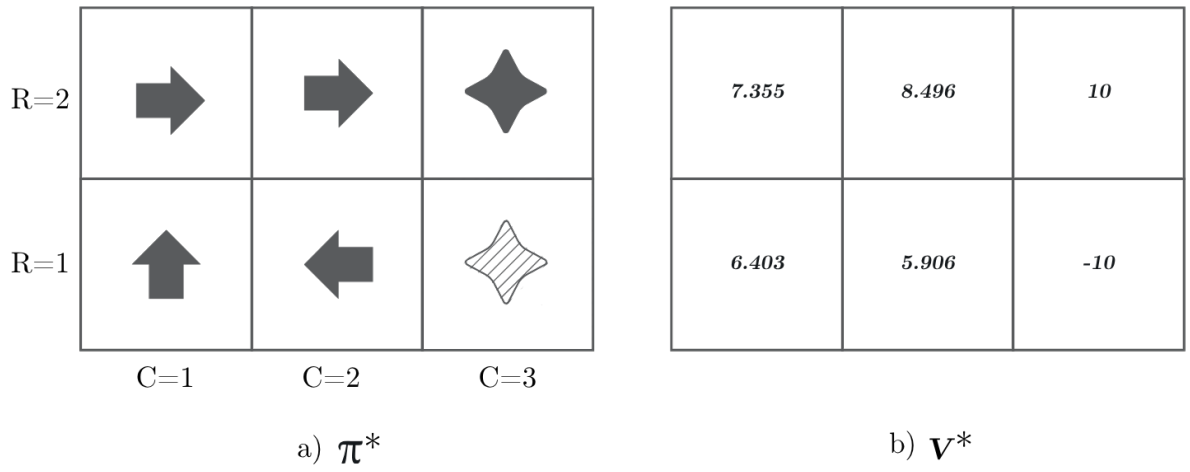

Figura 3: Política óptima del grid 2×3. (a) Acción óptima $\pi^*(r,c)$; (b) valor $V^*(r,c)$ calculado por Iteración de Valor con $\gamma = 0.9$, $\varepsilon = 0.1$.

La Iteración de Valor opera sobre la enumeración explícita de S, por lo que la complejidad de cada respaldo es $O(|S|^2 \cdot |A|)$. A medida que el número de variables de estado crece, $|S|$ crece exponencialmente y el algoritmo deja de ser aplicable directamente. La Sección 3.8 aborda esta limitación con los MDPs Factorizados.

3.8. MDPs Factorizados y Fluentes de Estado

Un MDP con n variables de estado tiene un espacio de estados de tamaño $\prod_{i=1}^{n} |\mathrm{dom}(X_i)|$. Para n variables binarias, $|S| = 2^n$. Con decenas de variables, la enumeración explícita de S es computacionalmente inviable. Los MDPs Factorizados evitan esta enumeración al codificar la dinámica en la estructura de las variables de estado [12].

En un MDP Factorizado, el estado s es una valuación del conjunto de variables $\mathbf{X} = (X_1, \ldots, X_n)$; cada X_i tiene dominio finito $\mathrm{dom}(X_i)$ [12]. La función de transición se factoriza según la estructura de una Red Bayesiana Dinámica (DBN). Para cada variable X_i, el valor siguiente X_i' depende únicamente de sus variables padre $\mathrm{Pa}(X_i') \subseteq \mathbf{X}$:

$$P(s' \mid s, a) = \prod_{i=1}^{n} P\big(X_i' \mid \mathrm{Pa}(X_i'),\, a\big) \tag{9}$$

Esta factorización expresa la dinámica del sistema en términos de dependencias locales. El número de parámetros necesarios crece linealmente con el número de variables en lugar de cuadráticamente con $|S|$.

En MDP-ProbLog, las variables del MDP Factorizado se denominan fluentes de estado. Un fluente de estado es una variable con un dominio discreto finito [4]. Un fluente booleano tiene dominio $\{0, 1\}$. Un fluente multivaluado tiene dominio $\{v_1, \ldots, v_k\}$ con $k \geq 2$. La Sección 3.9 describe cómo MDP-ProbLog integra estos fluentes con la programación lógica probabilística.

3.9. El framework MDP-ProbLog

MDP-ProbLog integra los MDPs Factorizados y el lenguaje ProbLog para resolver problemas de planificación probabilística de horizonte infinito. El framework descansa en la correspondencia entre la evaluación de un programa ProbLog bajo evidencia y el respaldo de Bellman sobre un estado del MDP [4]. El usuario especifica el modelo declarativamente en ProbLog; el solver transforma esa especificación en una política óptima por Iteración de Valor sobre el circuito compilado.

Un programa MDP-ProbLog extiende la sintaxis de ProbLog con tres categorías de predicados reservados [4]. El predicado `state_fluent/1` declara los fluentes de estado cuya valuación define un estado del MDP. El predicado `action/1` declara las acciones disponibles en cada estado. El predicado `utility/2` encapsula la función de recompensa; asigna a cada átomo un valor numérico.

La dinámica del MDP se especifica en un programa de dos instantes temporales. Los átomos con $t = 0$ representan el estado actual y la acción ejecutada; los de $t = 1$ representan el estado siguiente. Las reglas de transición conectan ambos instantes con hechos probabilísticos auxiliares que codifican la incertidumbre de las acciones.

El Código 4 exhibe la estructura de un programa MDP-ProbLog para el grid de la Sección 3.1

con codificación booleana. Los fluentes `row2(0)`, `col2(0)` y `col3(0)` codifican la posición del agente en tres bits.

```
% Fluentes de estado: codificacion booleana de la posicion del
   agente
% (3 bits generan 2^3 = 8 combinaciones; solo 6 corresponden a
   celdas reales)
state_fluent(row2(0)).    % fila 2 si verdadero, fila 1 si falso
state_fluent(col2(0)).    % columna 2 si verdadero
state_fluent(col3(0)).    % columna 3 si verdadero

% Acciones
action(norte). action(sur). action(este). action(oeste).

% Incertidumbre estocastica para la accion norte
0.8::avance_n(0).   0.1::desvio_e_n(0).   0.1::desvio_o_n(0).

% Transicion: avance hacia la fila 2 con accion norte (regla
   representativa)
row2(1) :- norte(0), avance_n(0), \+ row2(0).
% ... (reglas de transicion para col2, col3 y las demas acciones)

% Recompensa: +10 al alcanzar la celda meta (fila 2, columna 3)
meta(1) :- row2(1), col3(1), \+ col2(1).
utility(meta(1), 10).
```

Código 4: Estructura de un programa MDP-ProbLog para el grid 2×3 con codificación booleana.

Bueno et al. demuestran que evaluar un programa MDP-ProbLog bajo evidencia (s, a) produce las probabilidades de transición $P(s' \mid s, a)$ requeridas por el respaldo de Bellman [4]. La evaluación se realiza por WMC sobre el circuito compilado. Esta equivalencia garantiza que el algoritmo de IV de MDP-ProbLog calcula la misma política óptima que la IV clásica.

El solver de MDP-ProbLog ejecuta tres etapas [4]. En la etapa de preprocesamiento, se resuelven las reglas intencionales y se construye el programa aumentado con los predicados de función de valor. En la etapa de compilación, el programa aterrizado se convierte a una fórmula booleana ponderada y se compila a un circuito d-DNNF. En la etapa de Iteración de Valor, el solver fija la evidencia (s, a) para cada par estado-acción, evalúa el circuito por WMC y actualiza V hasta convergencia.

La restricción booleana de la versión original reside en la etapa de preprocesamiento. Cada fluente de estado se inyecta como el hecho probabilístico `0.5::f(0)`, que presupone un dominio binario $\{0, 1\}$. Esta decisión impide representar directamente variables categóricas sin codificación binaria manual. La extensión propuesta interviene exclusivamente en esta capa de inyección. En los fluentes multivaluados, sustituye los hechos probabilísticos independientes por Disyunciones Anotadas. El resto del pipeline permanece sin modificaciones.

4. Diseño e Implementación

El capítulo anterior estableció los formalismos que fundamentan la extensión: la Programación Lógica Probabilística, las Disyunciones Anotadas y el Conteo de Modelos Ponderados. Este capítulo transforma esos conceptos en artefactos de software. La Sección 4.1 analiza la arquitectura original; la Sección 4.8 demuestra la retrocompatibilidad con los modelos booleanos existentes. Las Secciones 4.2 a 4.7 describen el esquema factorizado del espacio de estados, el clasificador automático de fluentes, el *pipeline* de preparación del MDP, la adaptación del algoritmo de Iteración de Valor, el evaluador de Darwiche y los módulos de soporte.

4.1. Análisis de la arquitectura original

El punto de partida de cualquier extensión es comprender la arquitectura que se extiende. MDP-ProbLog opera bajo un diseño modular que separa la inferencia probabilística, la representación del espacio de estados, la orquestación del MDP y la resolución numérica en módulos con responsabilidades delimitadas [4]. El análisis de esta arquitectura permite identificar dónde reside la restricción booleana y qué módulos requiere modificar la extensión.

4.1.1. Componentes del sistema original

La arquitectura original separa las responsabilidades del *framework* en cuatro módulos. Cada módulo tiene una función delimitada y las dependencias fluyen en una sola dirección. La Figura 4 exhibe esta estructura; la responsabilidad de cada módulo se detalla a continuación.

`engine.py` Implementa un adaptador sobre el motor interno de ProbLog (`DefaultEngine`). Encapsula la Base de Datos de Cláusulas (`ClauseDB`) y expone métodos para añadir hechos lógicos, ejecutar el *grounding* relevante y compilar circuitos evaluables para el Conteo de Modelos Ponderados.

`fluent.py` Define las abstracciones del estado del sistema. Las clases `Fluent` y `StateSpace` generan e indexan las configuraciones posibles del espacio de estados, asumiendo un dominio factorizado puramente booleano.

`mdp.py` Actúa como coordinador del *framework*. Interroga la `ClauseDB` buscando predicados reservados (`state_fluent/1`, `action/1`, `utility/2`), coordina la incorporación de hechos temporales y traduce las relaciones lógicas en una semántica de transiciones de Markov.

`value_iteration.py` Implementa el algoritmo de Iteración de Valor. Opera sobre índices enteros y distribuciones marginales para calcular la función de valor óptima mediante la ecuación de optimalidad de Bellman.

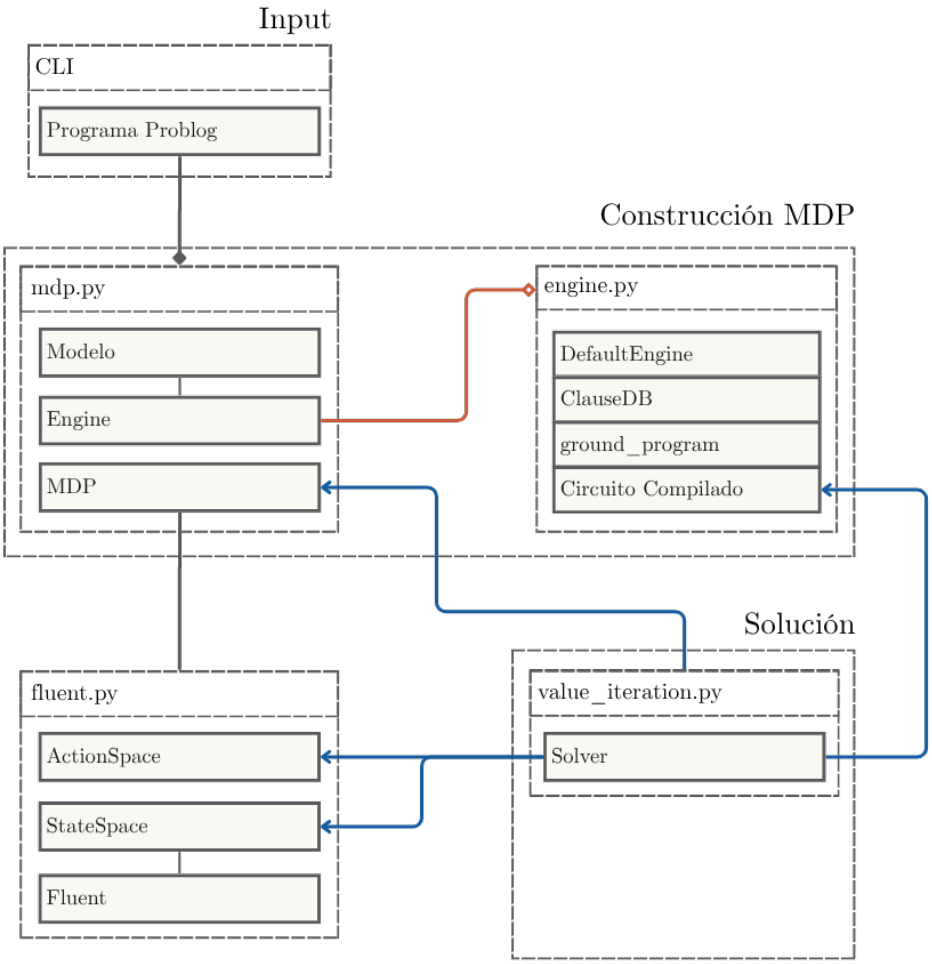

Figura 4: Diagrama de la arquitectura original del framework MDP-ProbLog.

El módulo `mdp.py` depende de `engine.py`, `fluent.py` y `value_iteration.py`. Los módulos `engine.py` y `fluent.py` son independientes entre sí. Esta organización delimita el alcance de la extensión: `fluent.py` y `mdp.py` son los candidatos a modificación; `engine.py` permanece inalterado.

4.1.2. Flujo de ejecución

La interacción entre estos módulos sigue un flujo de cinco fases:

1. **Análisis y carga.** El módulo `engine.py` realiza el *parsing* del archivo Prolog proporcionado por el usuario y lo inyecta en la `ClauseDB`.

2. **Descubrimiento del dominio.** El coordinador `mdp.py` interroga la `ClauseDB` para identificar los fluentes de estado y las acciones declaradas, e instancia los iteradores correspondientes en `fluent.py`.

3. **Inyección temporal.** Para modelar la dinámica secuencial, `mdp.py` convierte las declaraciones abstractas en versiones con marca de tiempo ($t = 0$ para el estado actual, $t = 1$ para el siguiente estado). En esta fase se introducen hechos probabilísticos neutrales en $t = 0$ que serán reemplazados por evidencia determinista durante la evaluación de cada par estado-acción.

4. **Aterrizado y compilación.** Se ejecuta un *grounding* relevante respecto a las consultas de interés. Las reglas de transición se compilan en un circuito proposicional evaluable (SDD o d-DNNF, según el *backend* seleccionado).

5. **Resolución.** El módulo `value_iteration.py` recorre el espacio de estados, inyecta cada par estado-acción como evidencia en el motor y solicita las probabilidades marginales $P(s' \mid s, a)$ mediante WMC para actualizar la función de valor $V(s)$.

Dos de estas cinco fases asumen implícitamente un dominio booleano. La Fase 3 inyecta hechos probabilísticos independientes para cada fluente, presuponiendo que cada variable de estado tiene dominio $\{0, 1\}$. La Fase 5 recorre un espacio de estados construido como el producto cartesiano de factores binarios. La extensión modifica estas dos fases y la estructura de datos que las sustenta.

4.1.3. Limitaciones en la gestión de fluentes

Forzar fluentes categóricos en variables de Bernoulli degrada el modelado lógico y la eficiencia computacional por la explosión de estados espurios, como se estableció en el Capítulo 3.

El diseño original implementa `StateSpace` como un iterador combinatorio plano que procesa las declaraciones `state_fluent/1` y genera mecánicamente el producto cartesiano del dominio binario sin considerar el agrupamiento ni la exclusión mutua.

La solución no requiere alterar el motor de inferencia de ProbLog. La compilación a circuitos y el WMC permanecen intactos. La intervención ocurre en la capa intermedia donde las declaraciones lógicas se transforman en variables de estado evaluables para el *solver*.

Se identifican tres puntos de extensión para integrar el soporte de fluentes multivaluados:

1. **Representación del espacio de estados.** Reemplazar `StateSpace` por una estructu-

ra factorizada que admita factores de cardinalidad arbitraria, eliminando la generación de combinaciones inválidas.

2. **Inyección de hechos en la `ClauseDB`.** Adaptar la Fase 3 del *pipeline* para que los fluentes multivaluados se representen como Disyunciones Anotadas en lugar de hechos probabilísticos independientes.

3. **Cálculo del valor esperado.** Reestructurar la evaluación del valor futuro en la Iteración de Valor para que recorra los factores del esquema en lugar de enumerar combinaciones binarias.

4.2. Esquema factorizado del espacio de estados

El primer componente a diseñar es la estructura de datos que reemplaza a `StateSpace`. La clase `FluentSchema` admite factores de cardinalidad arbitraria y expone operaciones uniformes de indexación y decodificación accesibles desde cualquier módulo del *framework*.

4.2.1. Modelo de datos

El `FluentSchema` organiza el espacio de estados como una secuencia ordenada de m factores independientes. Cada factor k es una lista de términos lógicos aterrizados que representan las opciones mutuamente excluyentes de esa variable de estado. La cardinalidad del factor k, denominada base b_k, es igual al número de opciones que contiene.

La clase mantiene tres estructuras internas:

1. **Lista de factores.** Una lista de listas de términos: el factor k contiene los b_k términos que representan sus opciones. La posición de cada término dentro de su factor determina su índice local $v_k \in [0, b_k - 1]$.

2. **Lista de bases.** Un arreglo de enteros $[b_1, b_2, \ldots, b_m]$ que registra la cardinalidad de cada factor. Los factores booleanos tienen base 2; los factores multivaluados tienen base $N \geq 2$.

3. **Lista aplanada.** La concatenación de todos los términos en orden de registro. Esta lista localiza cualquier término desde su índice global sin recorrer todos los factores.

Dos categorías de factor se distinguen semánticamente. Un **factor booleano** tiene base 2 y sus dos términos corresponden a los valores $\{0, 1\}$ de un fluente de Bernoulli. Un **factor multivaluado** tiene base $N \geq 2$ y sus N términos codifican los valores de una variable categórica en formato *one-hot*. Esta distinción determina la forma de inyección de hechos en el *pipeline* de preparación. Los factores booleanos se inyectan como hechos probabilísticos independientes; los multivaluados, como Disyunciones Anotadas.

4.2.2. Codificación *mixed-radix*

El `FluentSchema` asigna a cada configuración un índice entero $I \in [0, |\mathcal{S}|-1]$ con codificación de base mixta (*mixed-radix*). Esta codificación generaliza la codificación binaria; en lugar de base 2 uniforme, cada factor k usa su propia base b_k.

El tamaño total del espacio de estados es el producto de todas las bases:

$$|\mathcal{S}| = \prod_{k=1}^{m} b_k \tag{10}$$

Para mapear una configuración a un índice, se calcula un *stride* S_k por factor, definido como el producto de las bases de todos los factores que lo preceden:

$$S_k = \prod_{i=1}^{k-1} b_i \quad \text{con } S_1 = 1 \tag{11}$$

Dada una configuración donde el factor k toma el valor local $v_k \in [0, b_k - 1]$, el índice global se obtiene como:

$$I = \sum_{k=1}^{m} v_k \times S_k \tag{12}$$

La decodificación extrae el valor local v_k de cada factor como el residuo de I módulo b_k:

$$v_k = I \text{ mód } b_k, \quad I \leftarrow \left\lfloor \frac{I}{b_k} \right\rfloor \tag{13}$$

Ambas operaciones recorren los m factores en tiempo $O(m)$. La codificación garantiza que cada índice $I \in [0, |\mathcal{S}|-1]$ corresponde a exactamente una combinación semánticamente válida. Los estados espurios son imposibles: la codificación no genera índices para combinaciones inválidas.

4.2.3. Ejemplo mixed-radix

El ejemplo siguiente usa un dominio con dos variables de estado: **color**, con valores {`rojo`, `verde`, `azul`}, y **velocidad**, con valores {`lenta`, `rápida`}. El `FluentSchema` registra un factor de base 3 para color y uno de base 2 para velocidad.

El tamaño del espacio es $|\mathcal{S}| = 3 \times 2 = 6$, frente a los $2^5 = 32$ estados que generaría la codificación booleana equivalente con cinco fluentes independientes. Los *strides* son $S_1 = 1$ y $S_2 = 3$. La Tabla 7 lista los seis estados resultantes.

Tabla 7: Codificación *mixed-radix* para el dominio color × velocidad.

Índice I	**color** (v_1)	**velocidad** (v_2)	**Cálculo**
0	`rojo` (0)	`lenta` (0)	$0{\times}1 + 0{\times}3 = 0$
1	`verde` (1)	`lenta` (0)	$1{\times}1 + 0{\times}3 = 1$
2	`azul` (2)	`lenta` (0)	$2{\times}1 + 0{\times}3 = 2$
3	`rojo` (0)	`rápida` (1)	$0{\times}1 + 1{\times}3 = 3$
4	`verde` (1)	`rápida` (1)	$1{\times}1 + 1{\times}3 = 4$
5	`azul` (2)	`rápida` (1)	$2{\times}1 + 1{\times}3 = 5$

Por ejemplo, el estado con `azul` y `rápida` se codifica como $I = 2 \times 1 + 1 \times 3 = 5$. La decodificación parte de $I = 5$: el color se extrae como $5 \bmod 3 = 2$ (`azul`) y se actualiza $I \leftarrow \lfloor 5/3 \rfloor = 1$; la velocidad se extrae como $1 \bmod 2 = 1$ (`rápida`). La decodificación concluye al agotar los factores.

4.2.4. Operaciones relevantes

Dos operaciones del `FluentSchema` son invocadas recurrentemente por el resto del *framework*:

`get_factors_at(timestep)` Retorna la lista de factores con los términos marcados con la estampa de tiempo indicada. `mdp.py` la invoca para obtener los factores del estado actual ($t = 0$) y del siguiente ($t = 1$) durante la preparación del *pipeline*.

El marcado temporal agrega el argumento entero t como último argumento de cada término; el término `color(rojo)` a tiempo $t = 1$ se convierte en `color(rojo, 1)`.

`get_local_index(k, term)` Dado el índice de factor k y un término lógico aterrizado, retorna su posición v_k dentro de ese factor. La Sección 4.5 la invoca durante el recorrido recursivo de los factores. En cada nivel, traduce el término retornado por el evaluador a su posición v_k para acumular el índice global del estado sucesor.

El `FluentSchema` centraliza la gestión del espacio de estados; los demás módulos acceden a él exclusivamente por estas dos operaciones. El `FluentClassifier` (Sección 4.3) lee las declaraciones del programa lógico y construye el esquema de forma automática.

4.3. Clasificador automático de fluentes

El `FluentSchema` debe construirse antes de que el *framework* ejecute cualquier otra etapa. Su construcción requiere conocer, para cada fluente declarado, si pertenece a un factor booleano o a uno multivaluado. El predicado `state_fluent/1` declara la existencia del fluente, pero no su tipo. El módulo `FluentClassifier` obtiene esa información de los nodos `choice` que las Disyunciones Anotadas del programa dejan registrados en la `ClauseDB`. El clasificador examina esos nodos, infiere el tipo de cada fluente y construye el esquema sin intervención del usuario.

4.3.1. Modos de declaración

El clasificador admite dos formas de declarar fluentes de estado. En el **modo implícito**, el usuario escribe `state_fluent(f(X))` y el *framework* infiere el tipo desde la estructura de la `ClauseDB`. En el **modo explícito**, el usuario escribe `state_fluent(f(X), multivalued)` o `state_fluent(f(X), bool)` e indica el tipo directamente. El modo explícito tiene prioridad. Si un fluente aparece en ambas formas, el *framework* emite una advertencia y conserva la declaración explícita.

El modo implícito es suficiente para la mayoría de los dominios. El modo explícito es necesario en dominios con familias de variables indexadas por entidad, como se discute al final de esta sección.

4.3.2. El índice invertido de Disyunciones Anotadas

La inferencia implícita depende de una estructura auxiliar denominada **índice invertido**. El clasificador recorre la tabla de nodos de la `ClauseDB` e identifica los nodos de tipo `choice`: las ramas estocásticas que ProbLog genera para cada Disyunción Anotada. Por cada nodo `choice`, extrae el término asociado y registra cada argumento constante como clave del índice, bajo el identificador del grupo AD al que pertenece ese nodo.

El resultado es un diccionario de la forma:

$$\mathcal{I} : \Sigma \to \mathcal{P}(\Gamma)$$

donde Σ es el conjunto de valores constantes y Γ es el conjunto de identificadores de grupos AD. $\mathcal{I}(v)$ reúne los identificadores de los grupos AD que produjeron el valor v.

La regla de transición siguiente ilustra la construcción del índice:

```
0.7::color(rojo,1); 0.2::color(verde,1); 0.1::color(azul,1)
    :- color(rojo,0), cambiar.
```

ProbLog procesa esta AD y registra tres nodos `choice` bajo el grupo g_1: uno para `color(rojo,1)`, uno para `color(verde,1)` y uno para `color(azul,1)`. El clasificador extrae los argumentos constantes de cada término y construye:

$$\mathcal{I}(\texttt{rojo}) = \{g_1\}, \quad \mathcal{I}(\texttt{verde}) = \{g_1\}, \quad \mathcal{I}(\texttt{azul}) = \{g_1\}$$

Este índice queda disponible antes de que comience la clasificación.

4.3.3. Pipeline de clasificación

El método `classify()` ejecuta seis pasos secuenciales para construir el `FluentSchema`:

1. **Validación de declaraciones explícitas.** Se verifica que cada tag de tipo en las declaraciones `state_fluent/2` sea `bool` o `multivalued`. Los tags inválidos se acumulan y se lanzan como un único error compuesto (`FluentDeclarationError`) para que el usuario pueda corregirlos todos de una vez.

2. **Registro de fluentes explícitos.** Cada par `(término, tag)` se almacena en el registro explícito con su tipo ya conocido. Estos fluentes quedan excluidos de la inferencia implícita.

3. **Registro de fluentes implícitos.** Los fluentes declarados con `state_fluent/1` se agrupan por `(functor, aridad)`. Para cada grupo, se invoca `_infer_fluent_type()` (Sección 4.3.4) para determinar si el fluente es booleano o multivaluado.

4. **Fusión de registros.** Los resultados de los pasos 2 y 3 se fusionan en un registro unificado. Las declaraciones explícitas sobrescriben las implícitas cuando coinciden en la misma clave.

5. **Separación por tipo.** El registro se recorre en orden alfabético. Los fluentes booleanos se añaden al esquema con `schema.add_bool(term)`. Los fluentes multivaluados se acumulan por *functor* en un buffer temporal.

6. **Validación de cardinalidad.** Cada grupo multivaluado acumulado en el paso 5 se verifica: si contiene menos de dos opciones, se lanza un `FluentCardinalityError`, pues un factor de exclusión mutua requiere al menos dos alternativas. Los grupos válidos se registran con `schema.add_group(terms)`.

4.3.4. Inferencia implícita de tipo

El paso 3 del *pipeline* delega la decisión de tipo en `_infer_fluent_type()`. La función clasifica como booleano cualquier fluente de aridad 0. Los fluentes proposicionales carecen de argumentos y son binarios por definición. Si el fluente tiene aridad ≥ 1, la función examina cada posición argumental p del término. Para cada posición, extrae el conjunto V_p de valores que ese argumento toma en los términos aterrizados e intersecta los grupos AD de cada valor. Una intersección no vacía indica que todos los valores de la posición p provienen de un

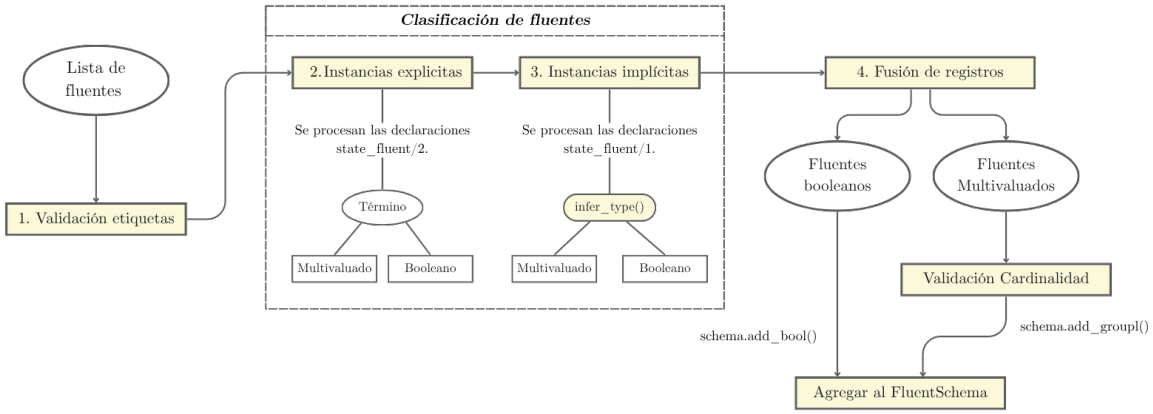

Figura 5: pipeline de clasificación. Se explica la ejecución del método `classify()`

mismo grupo AD; el fluente es multivaluado. Si ninguna posición pasa esta prueba, el fluente es booleano.

El Algoritmo 1 formaliza esta lógica.

Algoritmo 1 Inferencia de tipo por intersección de orígenes estocásticos

Entrada: Conjunto de términos aterrizados G de aridad a; índice invertido $\mathcal{I} : \Sigma \to \mathcal{P}(\Gamma)$.
Salida: Clasificación: `bool` o `multivalued`.

1: **si** $a = 0$ **entonces return** `bool`
2: **fin si**
3: **para** $p \leftarrow 1$ **a** a **hacer**
4: $\quad V_p \leftarrow \{\arg_p(g) \mid g \in G\}$
5: $\quad C \leftarrow \bigcap_{v \in V_p} \mathcal{I}(v)$
6: $\quad$ **si** $C \neq \emptyset$ **entonces return** `multivalued`
7: $\quad$ **fin si**
8: **fin para**
return `bool`

Aplíquese el algoritmo al fluente `color` del ejemplo anterior. Los términos aterrizados son `color(rojo)`, `color(verde)` y `color(azul)`, todos de aridad 1. En la posición 0, los valores son $V_0 = \{\texttt{rojo}, \texttt{verde}, \texttt{azul}\}$. La intersección es:

$$\mathcal{I}(\texttt{rojo}) \cap \mathcal{I}(\texttt{verde}) \cap \mathcal{I}(\texttt{azul}) = \{g_1\} \neq \emptyset$$

El algoritmo retorna `multivalued`. El fluente se registra como un factor de base 3 en el esquema.

El agrupamiento del paso 5 es correcto para variables categóricas globales. En esos dominios, todos los términos con el mismo *functor* corresponden a opciones de una única variable

de estado. Cuando múltiples entidades comparten predicado pero requieren independencia estocástica, el agrupamiento puede fusionar factores que deberían ser independientes. En esos casos, la declaración explícita con `state_fluent/2` define cada factor por separado y evita la consolidación automática.

Con el `FluentSchema` construido y validado, el *framework* dispone del espacio de estados necesario para el *pipeline* de preparación del MDP (Sección 4.4).

4.4. Pipeline de preparación del MDP

El *pipeline* de preparación transforma el `FluentSchema` en una base de cláusulas evaluable bajo condicionamiento. Esta transformación ocurre en el método `__prepare()` de la clase `MDP`, antes de que comience la Iteración de Valor.

4.4.1. Fases del pipeline

1. **Clasificación.** Se instancia el `FluentClassifier` para construir el `FluentSchema`. El resultado queda disponible para las fases siguientes.

2. **Inyección de hechos *dummy*.** Los hechos probabilísticos del estado actual ($t = 0$) se insertan en la `ClauseDB`, uno por factor del esquema. Estos hechos son *dummy*: sus probabilidades son valores neutrales que serán sustituidos en cada evaluación individual.

3. **Inyección de acciones.** Las acciones del modelo se inyectan como una Disyunción Anotada con distribución uniforme; la exclusión mutua queda garantizada por construcción.

4. **Aterrizado relevante.** Se ejecuta un *grounding* limitado a las consultas de interés: fluentes del siguiente estado ($t = 1$), utilidades, acciones y fluentes del estado actual ($t = 0$). Incluir los fluentes en $t = 0$ entre las consultas preserva sus identificadores en el circuito compilado; sin ellos, la sustitución de evidencia no podría localizar los nodos correctos.

5. **Compilación del circuito.** Los fluentes del siguiente estado y las utilidades se compilan en un único circuito d-DNNF o SDD. El circuito queda fijo y se reutiliza en todas las evaluaciones.

4.4.2. Hechos *dummy* y condicionamiento por evidencia

La Fase 2 inyecta hechos *dummy* en la `ClauseDB`. Su propósito deriva del modo en que el *framework* implementa el condicionamiento por evidencia.

Durante la Iteración de Valor, el *solver* evalúa $P(s' \mid s, a)$ para cada par estado-acción. Evaluar bajo un estado concreto significa fijar el valor de cada fluente en $t = 0$. ProbLog ofrece dos formas de condicionar evidencia: propagación lógica con el predicado `evidence/2` y sustitución directa de pesos en el circuito compilado, donde el literal verdadero recibe peso 1

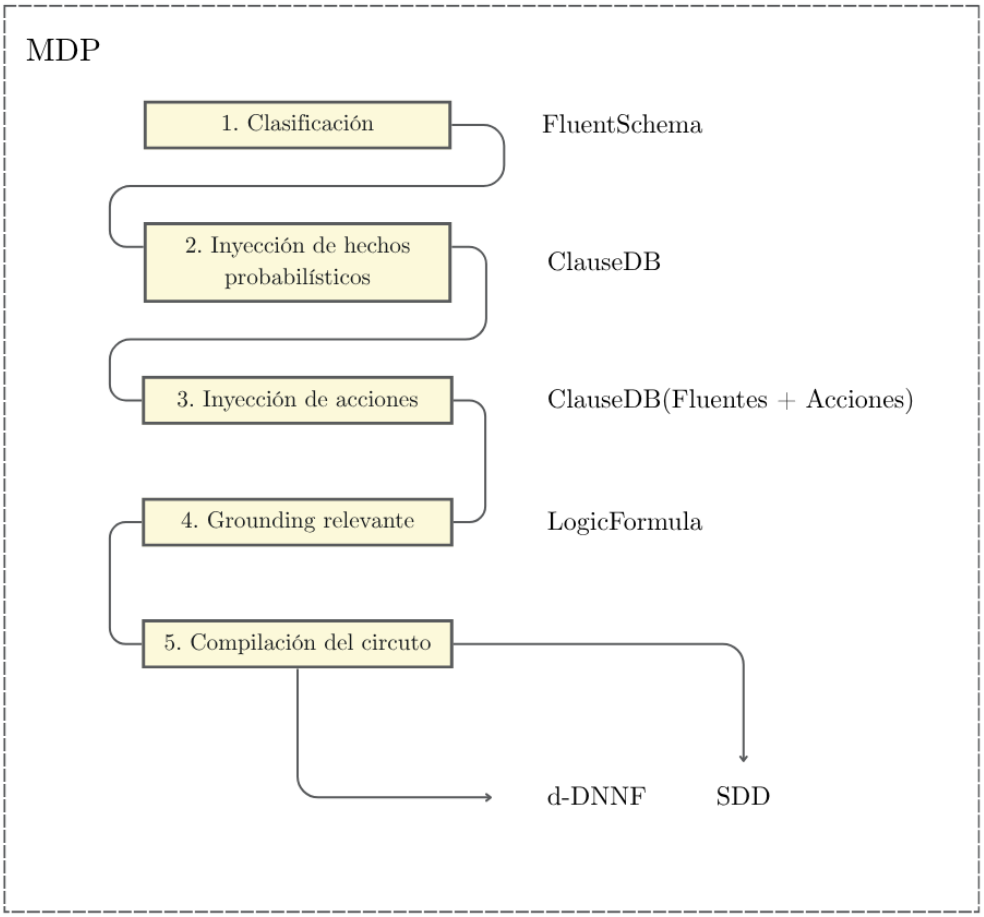

Figura 6: pipeline de preparación. Se explica la ejecución del método `__prepare()`

y el falso, peso 0. El *framework* usa la segunda vía porque opera sobre el circuito compilado y no recompila el programa.

La sustitución de pesos requiere que el nodo condicionado exista en el circuito como variable probabilística. Un hecho determinista carece de pesos y no puede condicionarse por esta vía. Los hechos *dummy* resuelven esta restricción: cada fluente de estado en $t = 0$ se inserta como hecho probabilístico antes de la compilación, y el compilador lo incluye como nodo con pesos en el circuito.

Los *dummies* se inyectan de forma distinta según el tipo de factor:

- **Factores booleanos.** Se inserta un hecho con probabilidad 0.5. Para el fluente `vivo`, la inyección produce `0.5::vivo(0)`.
- **Factores multivaluados.** Se inserta una Disyunción Anotada con distribución uniforme $1/N$ sobre las N opciones. Para el fluente `color` con tres valores, la inyección produce:

$$\tfrac{1}{3}\texttt{::color(rojo,0)}\ ;\ \tfrac{1}{3}\texttt{::color(verde,0)}\ ;\ \tfrac{1}{3}\texttt{::color(azul,0)}.$$

Las probabilidades neutrales de los *dummies* no afectan los resultados; se reemplazan antes de cada consulta. El evaluador condiciona `color = rojo` asignando peso $(1.0,\ 0.0)$ al nodo `color(rojo,0)` y peso $(0.0,\ 1.0)$ a los demás. El circuito calcula entonces $P(s' \mid \texttt{color=rojo}, a)$ sin recompilarse.

4.4.3. Corrección de la compilación doble

La versión original compilaba los fluentes del siguiente estado ($t = 1$) y las utilidades en invocaciones separadas. Cada llamada sobrescribía el circuito almacenado en `Engine`. En consecuencia, el mapa de nodos de los fluentes del siguiente estado quedaba sobreescrito por el de las utilidades.

La extensión corrige este problema con una compilación unificada. Los fluentes del siguiente estado y las utilidades quedan mapeados al mismo circuito desde el inicio, y ese circuito permanece inalterado durante toda la Iteración de Valor.

4.4.4. Declaración de fluentes multivaluados

La extensión del *pipeline* habilita una sintaxis de modelado más directa para variables categóricas. El Código 5 exhibe el modelo original para la variable `color` con tres valores.

```
% Tres fluentes independientes
state_fluent(color_rojo).
state_fluent(color_verde).
state_fluent(color_azul).

% Transicion desde color=rojo bajo la accion "cambiar"
```

```
% (el modelador es responsable de garantizar exclusion mutua)
0.7::color_rojo(1) :- color_rojo(0), cambiar.
0.2::color_verde(1) :- color_rojo(0), cambiar.
0.1::color_azul(1) :- color_rojo(0), cambiar.
```

Código 5: Modelado booleano de una variable categórica con tres valores.

Con la codificación booleana, el espacio de estados para esta sola variable tiene $2^3 = 8$ combinaciones. El Código 6 presenta el modelo equivalente con la extensión.

```
% Una declaracion para las tres opciones
state_fluent(color(X), multivalued) :- val(X).
val(rojo). val(verde). val(azul).

% Transicion con AD: exclusion mutua y suma a 1 garantizadas
0.7::color(rojo,1) ; 0.2::color(verde,1) ; 0.1::color(azul,1)
    :- color(rojo,0), cambiar.
```

Código 6: Modelado de la misma variable con la extensión multivaluada.

La declaración multivaluada produce un único factor de base 3 por lo que su espacio de estados tiene exactamente 3 combinaciones. La Disyunción Anotada de la regla de transición garantiza por construcción que exactamente una opción es verdadera en cada estado sucesor y que la suma de probabilidades es 1. No se requieren restricciones adicionales de exclusión mutua.

4.5. Adaptación de la Iteración de Valor

La codificación *mixed-radix* del espacio de estados exige que el cálculo del valor esperado recorra la estructura factorizada. En el diseño original, ese cálculo enumeraba las 2^n combinaciones binarias del espacio. La nueva implementación recorre los factores del esquema de forma recursiva y explota la independencia condicional entre ellos para evitar la enumeración explícita.

La ecuación de Bellman requiere evaluar, para cada par (s, a):

$$\mathbb{E}_{s'}[V(s') \mid s, a] = \sum_{s'} P(s' \mid s, a)\, V(s') \tag{14}$$

Cuando el espacio de estados está factorizado en m variables independientes, la distribución de transición se descompone como:

$$P(s' \mid s, a) = \prod_{k=1}^{m} P(x'_k \mid s, a) \tag{15}$$

donde $P(x'_k \mid s, a)$ es la distribución marginal del factor k. El evaluador devuelve directamente estas distribuciones marginales para cada factor. La suma sobre todos los estados sucesores equivale al producto cartesiano de estas distribuciones; el algoritmo lo recorre factor a factor.

Descripción del algoritmo. El recorrido comienza en el factor $k = 0$ con el índice global acumulado $I = 0$ y la probabilidad conjunta $P = 1$. En cada llamada recursiva al nivel k, el algoritmo itera sobre las ramas del factor k, es decir, los pares (v, p) donde v es un valor posible del factor y p su probabilidad marginal. Para cada rama activa (con $p > \epsilon$), el algoritmo:

1. Obtiene el índice local v_k de ese valor dentro del factor k con `get_local_index`.
2. Actualiza el índice global acumulado con la contribución del factor: $I' = I + v_k \times S_k$.
3. Actualiza la probabilidad conjunta del camino: $P' = P \times p$.
4. Recurre al nivel $k + 1$ con I' y P'.

Cuando $k = m$ (todos los factores han sido procesados), el índice acumulado I identifica el estado sucesor s' en la codificación *mixed-radix*. El algoritmo retorna $P \times V[I]$, la contribución de ese camino al valor esperado. La suma de estas contribuciones sobre todos los caminos produce $\mathbb{E}[V(s') \mid s, a]$.

Algoritmo 2 Valor esperado del estado sucesor sobre el esquema factorizado

Entrada: Factor actual k; índice *mixed-radix* acumulado I; probabilidad conjunta del camino P.
Entrada: Distribuciones marginales por factor $\mathcal{F} = [\mathcal{F}_0, \ldots, \mathcal{F}_{m-1}]$, donde cada $\mathcal{F}_k$ es una lista de pares $(valor,\ prob)$.
Entrada: *Strides* $S = [S_0, \ldots, S_{m-1}]$; función de valor V; umbral de poda ϵ.
Salida: Valor esperado escalar $\mathbb{E}[V(s') \mid s, a]$ para el subárbol.

```
function ValorEsperado(k, I, P)
    si k = m entonces                                  ▷ Caso final. return P × V[I]
    fin si
    acum ← 0
    para cada (v, p) ∈ F_k hacer
        si p > ε entonces                ▷ Ignorar ramas con probabilidad despreciable.
            v_k ← ÍndiceLocal(k, v)                    ▷ Posición de v dentro del factor k.
            I' ← I + v_k × S_k                ▷ Contribución de este factor al índice global.
            acum ← acum + ValorEsperado(k + 1, I', P × p)
        fin si
    fin parareturn acum
fin function
```

Ejemplo de ejecución. Retómese el dominio con dos factores: **color** (base 3, strides $S_0 = 1$) y **velocidad** (base 2, stride $S_1 = 3$). Supóngase que el par estado-acción produce las siguientes distribuciones marginales: $\mathcal{F}_0 = [(\texttt{rojo},\ 0.7),\ (\texttt{verde},\ 0.2),\ (\texttt{azul},\ 0.1)]$ y $\mathcal{F}_1 = [(\texttt{lenta},\ 0.6),\ (\texttt{rápida},\ 0.4)]$.

La llamada inicial es ValorEsperado$(0,\ 0,\ 1.0)$. En el nivel $k = 0$ se exploran tres ramas;

para la rama `rojo`: $v_0 = 0$, $I' = 0 + 0 \times 1 = 0$, se recurre a ValorEsperado$(1,\ 0,\ 0.7)$. En el nivel $k = 1$, rama `lenta`: $v_1 = 0$, $I' = 0 + 0 \times 3 = 0$, se retorna $0.7 \times 0.6 \times V[0]$. Rama `rápida`: $v_1 = 1$, $I' = 0 + 1 \times 3 = 3$, se retorna $0.7 \times 0.4 \times V[3]$. El recorrido continúa para las ramas `verde` y `azul` del nivel $k = 0$. La suma de todas las contribuciones produce el valor esperado completo en exactamente $|\mathcal{F}_0| \times |\mathcal{F}_1| = 6$ hojas, que coincide con el número de estados válidos del espacio.

La poda por umbral ϵ reduce este recorrido en transiciones cuasi-deterministas. Si una sola rama por factor concentra casi toda la masa probabilística, el árbol efectivo tiene profundidad m y solo 1 hoja, frente a las $\prod_k |\mathcal{F}_k|$ hojas del caso general.

4.6. Evaluador de Darwiche

Tras la Fase 5 del *pipeline*, el circuito queda compilado y disponible para su evaluación. En cada par estado-acción, el solver lanza Q consultas independientes sobre ese circuito. Cada consulta corresponde a un fluente del estado sucesor ($t = 1$) o a un término de utilidad. El evaluador estándar recorre el circuito completo por cada consulta; con $Q = 50$ consultas activas, el costo se multiplica por 50 aunque todas compartan la misma evidencia. Esta sección describe el evaluador alternativo de Darwiche (2003), que resuelve las Q consultas en exactamente dos recorridos del circuito, independientemente del valor de Q.

4.6.1. El costo de la evaluación estándar

El evaluador estándar de ProbLog (`SimpleDDNNFEvaluator`) dedica un recorrido completo del circuito a cada consulta, con costo $O(|C|)$ donde $|C|$ es el número de nodos. Cuando el motor necesita resolver Q consultas simultáneas, ejecuta Q recorridos independientes, con un costo total de $O(Q \times |C|)$ por par estado-acción.

Cada par estado-acción requiere una consulta por cada fluente del estado siguiente más una por cada término de utilidad. Para un modelo con m factores de estado y u utilidades, el valor de Q es $m + u$. Si el circuito tiene $|C|$ nodos y el espacio de búsqueda tiene $|\mathcal{S}| \times |\mathcal{A}|$ pares, el costo total de la Iteración de Valor es:

$$O(|\mathcal{S}| \times |\mathcal{A}| \times Q \times |C|) \tag{16}$$

El factor Q multiplica el trabajo por el número de consultas en cada evaluación. A medida que el número de factores de estado crece, Q crece con él, y el costo de cada iteración de Bellman aumenta proporcionalmente.

4.6.2. La idea central: derivadas como respuestas

La redundancia del evaluador estándar tiene una causa precisa. Cada uno de los Q recorridos recorre el circuito entero para extraer un único valor numérico, descartando la información estructural acumulada en los demás nodos. Si se pudiera preservar esa información entre consultas, bastaría con un recorrido único para responder todas las consultas de golpe.

Darwiche observó que el circuito compilado representa un polinomio multilineal $F(\lambda_1, \ldots, \lambda_n)$, donde cada λ_i es una variable indicadora asociada a un literal del circuito. Bajo esta interpretación, la probabilidad marginal de una consulta q satisface:

$$\Pr(q \mid e) = \frac{w_q \cdot \partial F / \partial \lambda_q}{F(e)} \tag{17}$$

donde w_q es el peso probabilístico del literal q y $F(e)$ es el valor del circuito bajo la evidencia e. Obtener las Q marginales equivale a calcular Q derivadas parciales del mismo polinomio. La ventaja es que todas las derivadas parciales pueden calcularse en exactamente dos recorridos lineales del circuito, independientemente de cuántas consultas haya.

4.6.3. El algoritmo de diferenciación en dos pasadas

El algoritmo asocia a cada nodo i del circuito dos cantidades: $\mathbf{val}(i)$, calculada de hojas a raíz, y $\mathbf{pd}(i)$, calculada de raíz a hojas. La primera es el valor de evaluación ascendente del nodo; la segunda es la derivada parcial de F respecto de ese valor, que cuantifica cuánto cambia la salida del circuito al modificar $\mathrm{val}(i)$.

Pasada ascendente. Se recorren los nodos en orden topológico desde las hojas hasta la raíz. El valor de cada nodo se calcula según su tipo:

- **Hoja** (literal λ_i con peso w_i): $\mathrm{val}(i) = w_i$. Si la evidencia activa hace el literal falso, $\mathrm{val}(i) = 0$.
- **Conjunción** (nodo AND): $\mathrm{val}(i) = \prod_{j \in \mathrm{hijos}(i)} \mathrm{val}(j)$.
- **Disyunción** (nodo OR): $\mathrm{val}(i) = \sum_{j \in \mathrm{hijos}(i)} \mathrm{val}(j)$.

Al terminar, $\mathrm{val}(\text{raíz}) = F(e)$, el WMC del circuito bajo la evidencia actual.

Pasada descendente. Se recorren los nodos en orden topológico inverso desde la raíz hasta las hojas. La raíz recibe $\mathrm{pd}(\text{raíz}) = 1$. Para cada nodo i y cada hijo j:

- **Padre OR**: $\mathrm{pd}(j) \mathrel{+}= \mathrm{pd}(i)$. La derivada del padre se transmite íntegra a cada hijo porque $\partial\,(\mathrm{val}(c_1) + \mathrm{val}(c_2)) / \partial\,\mathrm{val}(c_j) = 1$.
- **Padre AND**: $\mathrm{pd}(j) \mathrel{+}= \mathrm{pd}(i) \times \prod_{k \in \mathrm{hijos}(i),\, k \neq j} \mathrm{val}(k)$. La derivada del padre se escala por el producto de los valores de los demás hijos porque $\partial \left(\prod_k \mathrm{val}(c_k)\right) / \partial\,\mathrm{val}(c_j) = \prod_{k \neq j} \mathrm{val}(c_k)$.

El acumulado $+=$ es necesario cuando un nodo tiene múltiples padres (circuitos tipo DAG): la regla de la cadena suma las contribuciones de cada padre.

Extracción de marginales. Una vez completadas ambas pasadas, la probabilidad marginal de cualquier consulta q asociada al nodo n_q se obtiene en $O(1)$:

$$\Pr(q \mid e) = \frac{w_q \cdot \mathrm{pd}(n_q)}{F(e)} \tag{18}$$

El Algoritmo 3 resume las dos pasadas y la extracción.

Algoritmo 3 Diferenciación de circuitos d-DNNF (Darwiche, 2003)

Entrada: Circuito con n nodos en orden topológico; pesos w_i ajustados por evidencia.
Salida: val[i] y pd[i] para todo nodo i; marginal de cualquier consulta en $O(1)$.

```
    // Pasada ascendente
 1: para i ← 1 a n hacer
 2:     si i es hoja entonces val[i] ← w_i
 3:     si no si i es AND entonces val[i] ← ∏_{j∈hijos(i)} val[j]
 4:     si no  (i es OR) val[i] ← ∑_{j∈hijos(i)} val[j]
 5:     fin si
 6: fin para
    // Pasada descendente
 7: pd[raíz] ← 1
 8: para i ← n a 1 hacer
 9:     para cada hijo j de i hacer
10:         si i es OR entonces pd[j] += pd[i]
11:         si no  (i es AND) pd[j] += pd[i] × ∏_{k∈hijos(i), k≠j} val[k]
12:         fin si
13:     fin para
14: fin para
    // Extracción de consultas
15: para cada consulta q con nodo n_q y peso w_q hacer
16:     Pr(q | e) ← w_q · pd[n_q] / val[raíz]
17: fin para
```

Ilustración del paso descendente. Para concretar la regla del nodo AND, considérese un nodo con dos hijos c_1 y c_2 con valores $\mathrm{val}(c_1) = 0.7$ y $\mathrm{val}(c_2) = 0.6$. La pasada ascendente produce $\mathrm{val}(\mathrm{AND}) = 0.42$. Si $\mathrm{pd}(\mathrm{AND}) = 1$, la pasada descendente asigna:

$$\mathrm{pd}(c_1) = 1 \times 0.6 = 0.6 \qquad \mathrm{pd}(c_2) = 1 \times 0.7 = 0.7$$

El mensaje que recibe c_1 es el valor de su hermano c_2: si c_1 aumentase en una unidad, el producto aumentaría en 0.6 unidades, que es exactamente la derivada parcial del producto respecto de c_1. La regla del nodo OR es más directa: ambos hijos reciben el gradiente del padre sin modificar, pues la derivada de una suma respecto de cualquier sumando es siempre 1.

4.6.4. Implementación y activación

La implementación separa la estructura del circuito de su evaluación en dos clases.

`DDNNFTopology` es una caché inmutable que almacena el tipo de cada nodo, su lista de hijos y su número total. Se calcula una sola vez por circuito compilado y se comparte entre todas las evaluaciones posteriores. Su construcción verifica que los nodos estén en orden topológico estricto: para todo nodo interno, todos sus hijos tienen índice menor. Esta propiedad, garantizada por los compiladores de ProbLog, hace que ambas pasadas se reduzcan a bucles simples sin ordenación explícita.

`DarwicheDDNNFEvaluator` es una subclase de `problog.evaluator.Evaluator` que implementa la interfaz estándar de ProbLog con el evaluador de diferenciación. Su método `propagate()` aplica la evidencia, ejecuta la pasada ascendente y luego la descendente. Su método `evaluate_all_queries(` extrae todas las marginales en $O(Q)$ según la Ecuación 18, sin ningún recorrido adicional del circuito.

El evaluador se activa con el parámetro `darwiche=True` al instanciar el `MDP`. Su uso está restringido al *backend* d-DNNF; no es compatible con SDD. La Tabla 8 resume la comparación de costo entre los dos evaluadores.

Tabla 8: Comparación de costo asintótico entre los evaluadores de ProbLog.

Operación	**Estándar**	**Darwiche**
Una sola consulta	$O(\|C\|)$	$O(1)$ tras propagación
Q consultas simultáneas	$O(Q \times \|C\|)$	$O(\|C\|) + O(Q)$
Iteración de Valor completa	$O(S \times A \times Q \times \|C\|)$	$O(S \times A \times \|C\|)$

La ganancia es proporcional a Q. Para un modelo con 8 consultas por par estado-acción ($Q = 8$), el costo de cada evaluación se reduce en un factor 8 respecto al evaluador estándar. En dominios con espacios de estados grandes o muchas utilidades, donde Q puede superar las decenas, esta reducción tiene un impacto directo en el tiempo de convergencia de la Iteración de Valor.

4.7. Herramientas auxiliares

Al converger la Iteración de Valor, el solver produce la función de valor óptima $V^*(s)$, la política óptima $\pi^*(s)$ y, si fue solicitada, la tabla $Q^*(s, a)$. Estas estructuras residen en memoria como objetos internos del *framework*. Para que el usuario pueda inspeccionarlas y validar la corrección del modelo resuelto, la extensión incorpora dos módulos de soporte: `CSVExporter` y `Simulator`.

4.7.1. Exportador de resultados

`CSVExporter` serializa las estructuras del MDP y los resultados de la Iteración de Valor a archivos en formato CSV. El exportador recibe como entradas el objeto `MDP` preparado y el objeto resultado de la Iteración de Valor. Expone un método por estructura exportada. Con esta organización, el usuario solicita únicamente las salidas que necesite en cada análisis. El método `export_all(vi)` actúa como orquestador e invoca todos los métodos en secuencia,

omitiendo las exportaciones opcionales cuando la estructura correspondiente no está disponible. Cada archivo generado comienza con un encabezado de metadatos formado por líneas con prefijo `#` que registran el tipo de contenido y la marca de tiempo de generación. La Tabla 9 lista los métodos y las estructuras que producen.

Tabla 9: Métodos de exportación del `CSVExporter` y estructuras producidas.

Método	**Estructura exportada**	**Disponibilidad**
`export_transition_matrix()`	$P(s' \mid s, a)$ plana	Siempre
`export_reward_matrix()`	$R(s, a)$	Siempre
`export_value_function(vi)`	$V^*(s)$	Siempre
`export_policy(vi)`	$\pi^*(s)$	Siempre
`export_q_table(vi)`	$Q^*(s, a)$	Si fue computada
`export_convergence(vi)`	Historial $V_k(s)$	Si fue registrado
`export_all(vi)`	Todas las anteriores	Orquestador

La exportación de la matriz de transición requiere un paso de expansión que las demás exportaciones no necesitan. El *framework* almacena transiciones en formato factorizado. Para cada par (s, a), el motor devuelve distribuciones marginales independientes por factor. La matriz $P(s' \mid s, a)$, en cambio, requiere la probabilidad conjunta sobre todos los estados sucesores. Para obtenerla, el método `_expand_transitions()` computa el producto cartesiano de las distribuciones factorizadas. Esta función aplica la misma recursión sobre factores descrita en la Sección 4.5: en cada nivel k se itera sobre las ramas del factor k, se calcula el índice global acumulado mediante `get_local_index` y el *stride* S_k, y se recurre al nivel $k+1$ con la probabilidad conjunta actualizada. Solo se emiten entradas con probabilidad estrictamente mayor que cero. Esta restricción mantiene el volumen del archivo reducido en modelos con transiciones dispersas.

4.7.2. Simulador de trayectorias

La política óptima $\pi^*(s)$ es un resultado analítico obtenido por programación dinámica. Una forma de verificar su calidad de forma independiente es ejecutarla contra la dinámica estocástica del modelo. El retorno acumulado que se obtiene en la práctica permite contrastar el resultado analítico con el comportamiento empírico. `Simulator` realiza esta verificación con rollouts.

`Simulator` recibe como entradas el MDP preparado y la política $\pi^* : \mathcal{S} \to \mathcal{A}$ como diccionario. El método principal es `run(trials, horizon, start_state, gamma)`. Ejecuta `trials` trayectorias independientes, cada una partiendo de `start_state`. Las recompensas se descuentan con el factor γ en cada paso y la trayectoria avanza hasta `horizon` pasos como máximo. La Tabla 10 describe los parámetros.

En cada paso de tiempo, el simulador ejecuta cuatro operaciones en secuencia:

1. Consulta $\pi^*(s)$ para determinar la acción a prescrita en el estado actual s.

Tabla 10: Parámetros del método `Simulator.run()`.

Parámetro	Tipo	Descripción
`trials`	`int`	Número de trayectorias independientes a ejecutar.
`horizon`	`int`	Longitud máxima de cada trayectoria en pasos.
`start_state`	`tuple`	Estado inicial desde el que parten todas las trayectorias.
`gamma`	`float`	Factor de descuento $\gamma \in (0,\ 1]$.

2. Invoca `structured_transition(s, a)` para obtener las distribuciones factorizadas $[\mathcal{F}_0, \ldots, \mathcal{F}_{m-1}]$.

3. Para cada factor k, muestrea el valor del estado sucesor por selección categórica ponderada sobre las ramas de $\mathcal{F}_k$.

4. Reconstruye la codificación del estado sucesor con `get_local_index`: los factores booleanos producen un valor en $\{0, 1\}$; los factores multivaluados producen la codificación *one-hot* del valor muestreado.

El muestreo opera factor a factor de forma independiente. Esta independencia es válida bajo el supuesto de independencia condicional entre factores establecido en la Ecuación 15: el valor sucesor de cada variable de estado es estadísticamente independiente del de las demás, dado el par (s, a). El método devuelve la recompensa promedio sobre todos los ensayos, la lista de recompensas individuales y las trayectorias completas. Las trayectorias individuales exponen episodios concretos, no solo el retorno promedio.

`CSVExporter` y `Simulator` operan sobre la misma estructura factorizada que utiliza el solver. Esta consistencia garantiza que la expansión de transiciones del exportador y el muestreo del simulador sean coherentes con los cálculos de la Iteración de Valor. La Sección 4.8 verifica que estas modificaciones preservan el comportamiento del *framework* sobre modelos booleanos existentes.

4.8. Retrocompatibilidad

La extensión fue diseñada para preservar el comportamiento del *framework* ante programas que utilizan exclusivamente fluentes booleanos, sin requerir modificaciones por parte del usuario. Esta garantía se sustenta en el comportamiento del `FluentClassifier` descrito en la Sección 4.3. Cuando el programa no contiene Disyunciones Anotadas, no hay nodos `choice` que registrar en el índice invertido. La clasificación implícita de todo fluente declarado con `state_fluent/1` resulta entonces en tipo booleano.

Bajo esta clasificación, el `FluentSchema` registra cada fluente como un factor de base 2. El cálculo de *strides* para m factores booleanos produce la serie $\{1,\ 2,\ 4,\ \ldots,\ 2^{m-1}\}$. Esta serie es la codificación binaria estándar de posiciones. La inyección de hechos *dummy*, descrita

en la Sección 4.4.2, inserta por cada fluente un hecho con probabilidad 0.5. El espacio de estados resultante tiene tamaño $|\mathcal{S}| = 2^m$ y se enumera como el producto cartesiano de m factores binarios. El comportamiento observado en cada etapa del *pipeline* es idéntico al de la arquitectura original.

La extensión también soporta modelos mixtos, donde algunos fluentes son booleanos y otros son multivaluados. El `FluentClassifier` trata cada tipo de forma independiente. Los fluentes booleanos se registran como factores de base 2 y los multivaluados como factores de base N. El usuario puede incorporar fluentes multivaluados en un programa existente añadiendo únicamente las nuevas declaraciones. Las reglas de transición booleanas no requieren modificación. Esta propiedad habilita una adopción gradual de la sintaxis multivaluada, sin reescribir las partes del programa que ya son correctas.

Con la arquitectura, los algoritmos y la implementación definidos, el sistema cuenta con la capacidad de modelar y resolver MDPs con fluentes multivaluados de manera nativa. La propiedad de retrocompatibilidad garantiza que esta extensión no invalida ningún modelo booleano existente. El siguiente capítulo somete la extensión a una evaluación empírica con casos de estudio que cuantifican su impacto tanto en la simplificación del modelado como en el comportamiento computacional del solver.

5. Experimentación y Resultados

Esta sección evalúa la extensión propuesta mediante experimentos diseñados para responder tres preguntas. ¿La codificación binaria y la codificación factorizada generan la misma función de valor óptima V^* y la misma política óptima π^* para un mismo MDP? ¿Reduce la codificación factorizada la complejidad del programa ProbLog que el modelador debe escribir respecto a la codificación binaria? ¿Cómo afecta la elección entre fluentes multivaluados o booleanos los tiempos de construcción del MDP y de resolución por Iteración de Valor?

5.1. Diseño experimental

5.1.1. Dominios de evaluación

El dominio primario es la familia de *grids* de Mitchell [9], presentada en la Sección 1.1. Sus transiciones deterministas permiten aislar el efecto de la codificación sobre el tamaño del programa y el espacio de estados. Se evalúan ocho tamaños, de 2×3 a 32×32 (Tabla 12); el 2×3 sirve como punto de verificación contra la solución publicada.

El dominio secundario es el *grid* canónico 3×4 de Russell y Norvig [23]: 11 estados, transiciones estocásticas (0.80/0.10/0.10), estado trampa en $(2, 4)$ con recompensa -1 y costo por paso de -0.04. Complementa al dominio Mitchell al verificar la corrección de la extensión bajo distribuciones probabilísticas no triviales. Se evalúa en una única instancia contra la Figura 17.3 de [23].

5.1.2. Variables independientes

El experimento manipula dos variables independientes. La primera es la **codificación del espacio de estados**: binaria o factorizada. La codificación binaria se aplica únicamente al dominio Mitchell; la factorizada se aplica a ambos dominios.

La segunda variable es la **configuración de inferencia**. ProbLog dispone de dos *backends* de compilación: circuitos d-DNNF [20] y Diagramas de Decisión Sentencial (SDD) [19]. Para los circuitos d-DNNF se comparan dos evaluadores: el estándar, que recorre el circuito una vez por consulta, y el diferencial de Darwiche [20], que calcula todas las marginales en un único recorrido por diferenciación. El SDD usa el evaluador estándar de ProbLog. A lo largo del capítulo, las tres configuraciones se identifican como *ddnnf*, *darwiche* y *sdd*, respectivamente.

5.1.3. Protocolo de medición

El experimento comprende 50 configuraciones: 48 del dominio Mitchell (8 tamaños $\times$ 2 codificaciones $\times$ 3 configuraciones de inferencia) y 2 del dominio Russell (codificación factorizada con *ddnnf* y darwiche). El pipeline completo se ejecuta en cada corrida desde cero: *parsing*, clasificación de fluentes, *grounding*, compilación del circuito e Iteración de Valor. Cada configuración se repite $N = 10$ veces de forma independiente.

El límite de tiempo es de 400 s por ejecución. Si una ejecución supera ese límite, se registra con estado `TIMEOUT` y los datos parciales disponibles. Los parámetros del solver se fijan en $\gamma = 0.9$ para el dominio Mitchell y $\gamma = 1.0$ para el dominio Russell, con $\varepsilon = 0.01$ y $\varepsilon_{\text{thr}} = 10^{-6}$. Los tiempos se miden con `time.perf_counter()` dentro de cada corrida.

5.2. Entorno de evaluación

Los experimentos se ejecutaron sobre un equipo Intel® Core™ Ultra 7 155U (14 núcleos) con 16 GB de RAM y sistema operativo Linux 6.17.0-20-generic. El intérprete fue Python 3.12.3. Las mediciones se realizaron en ejecuciones seriales; el proceso disponía de todos los recursos del equipo en cada corrida.

De las 50 configuraciones, 43 completaron las 10 corridas dentro del límite de *timeout*; las 7 restantes registraron `TIMEOUT` en todas sus ejecuciones (Tabla 13).

5.3. Corrección funcional

Esta sección verifica que la extensión multivaluada produce la misma V^* y la misma π^* que la codificación binaria para un mismo MDP, y que los resultados coinciden con la solución de referencia publicada.

5.3.1. Verificación en el dominio Mitchell

La Tabla 11 compara $V^*(s)$ y $\pi^*(s)$ para cada tamaño donde ambas codificaciones completaron. La columna $\max|\Delta V^*|$ reporta la diferencia máxima absoluta entre funciones de valor; la columna Iter. el número de iteraciones de Bellman hasta convergencia. La tolerancia de comparación es 10^{-6}.

Tabla 11: Verificación cruzada de corrección funcional en el dominio Mitchell.

Grid	$\lvert\mathcal{S}\rvert_{\text{bin}}$	$\lvert\mathcal{S}\rvert_{\text{fac}}$	$\max\lvert\Delta V^*\rvert$	π^* **coincide**	**Iter.**
2×3	8	6	0.0	Sí	3
3×3	16	9	0.0	Sí	3
4×4	32	16	0.0	Sí	4
6×6	64	36	0.0	Sí	6
8×8	128	64	0.0	Sí	8
12×12	256	144	0.0	Sí	12
16×16	512	256	0.0	Sí	16

El *grid* 32×32 no aparece porque la codificación binaria superó el *timeout* en las tres configuraciones de inferencia; la verificación cruzada requiere que ambas codificaciones completen.

Las diferencias son exactamente 0.0 en todos los tamaños evaluados y las políticas coinciden en cada caso. Ambas codificaciones representan el mismo MDP; la estructura de transiciones y recompensas es idéntica, por lo que la Iteración de Valor converge en el mismo número de pasos con los mismos parámetros γ y ε [3]. La verificación del dominio Russell se presenta en la Sección 5.6.

5.4. Simplificación del modelado

Esta sección evalúa la reducción de complejidad en el programa ProbLog. La Tabla 12 compara, para ambas codificaciones y los ocho tamaños de *grid*, el número de fluentes, las cardinalidades de sus dominios, el número de reglas, el tamaño del espacio de estados $|\mathcal{S}|$ y los estados espurios. Para la codificación binaria, la columna Cardinalidades indica el número de bits; para la factorizada, las cardinalidades de los dos fluentes multivaluados.

Tabla 12: Métricas del programa fuente por codificación y tamaño de *grid*.

Grid	**Cod.**	**Fluentes**	**Cardinalidades**	**Reglas**	$\lvert\mathcal{S}\rvert$	**Esp.**
2×3	Bin.	3	3 bits	42	8	2
	Fac.	2	2×3	21	6	0
3×3	Bin.	4	4 bits	77	16	7
	Fac.	2	3×3	21	9	0
4×4	Bin.	5	5 bits	178	32	16
	Fac.	2	4×4	21	16	0
6×6	Bin.	6	6 bits	472	64	28
	Fac.	2	6×6	21	36	0
8×8	Bin.	7	7 bits	1 025	128	64
	Fac.	2	8×8	21	64	0
12×12	Bin.	8	8 bits	2 595	256	112
	Fac.	2	12×12	21	144	0
16×16	Bin.	9	9 bits	5 376	512	256
	Fac.	2	16×16	21	256	0
32×32	Bin.	11	11 bits	26 623	2 048	1 024
	Fac.	2	32×32	21	1 024	0

La codificación factorizada mantiene 21 reglas para cualquier tamaño de *grid*; la binaria escala de 42 a 26 623 en el mismo rango. La codificación factorizada no genera estados espurios; la binaria los acumula con cada incremento de escala. La Figura 7 ilustra esta divergencia.

El *grid* 4×4 presenta una restricción adicional. Sus 16 celdas requieren 4 bits como mínimo, pero la implementación no puede usar el estado 0 para modelar transiciones, lo que fuerza el uso de 5 bits y genera 16 estados espurios. Esta restricción afecta cualquier *grid* cuyo número de celdas sea una potencia exacta de 2.

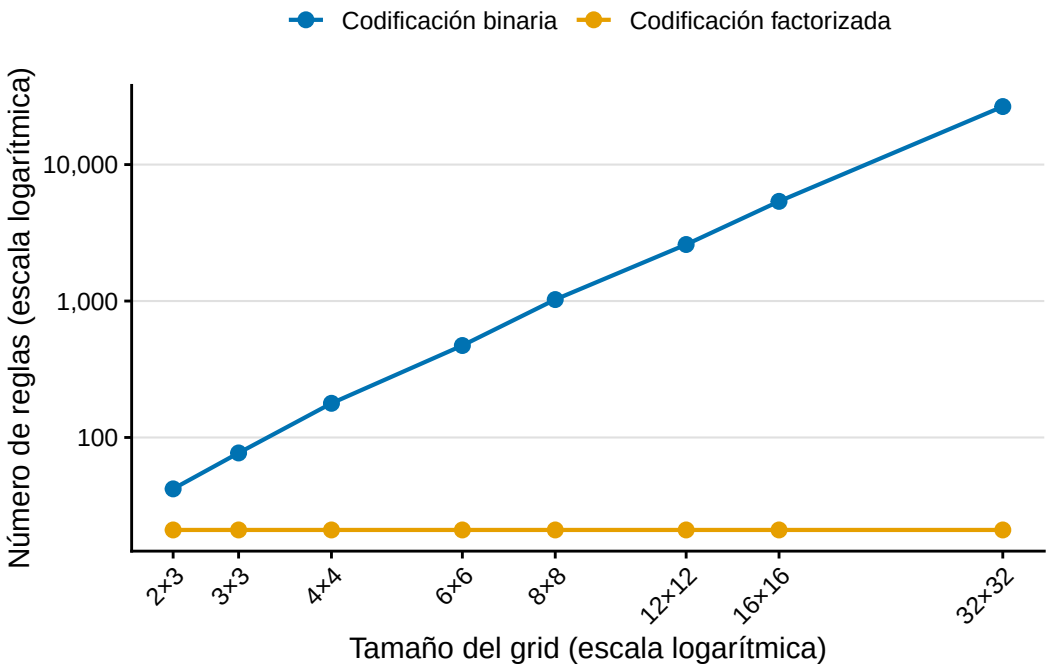

Figura 7: Número de reglas del programa fuente en función del número de celdas del *grid*.

5.5. Impacto computacional

Esta sección evalúa cómo influye la elección de codificación sobre el rendimiento del solver. La fase de construcción (t_{build}) abarca *parsing*, clasificación de fluentes, *grounding* y compilación del circuito. La fase de Iteración de Valor (t_{VI}) ejecuta las actualizaciones de Bellman hasta convergencia. El tiempo total es $t_{\text{total}} = t_{\text{build}} + t_{\text{VI}}$.

5.5.1. Tiempos de ejecución

La Tabla 13 presenta t_{total} para cada combinación de codificación y configuración de inferencia. Las configuraciones que completaron reportan media y desviación estándar sobre 10 ejecuciones independientes.

Tabla 13: Tiempo total de ejecución t_{total} (segundos) por codificación y configuración de inferencia.

Grid	**Cod.**	**ddnnf** (s)	**darwiche** (s)	**sdd** (s)
2×3	Bin.	0.218 ± 0.117	0.109 ± 0.005	0.056 ± 0.003
	Fac.	0.237 ± 0.014	0.103 ± 0.008	0.059 ± 0.018
3×3	Bin.	0.805 ± 0.037	0.379 ± 0.016	0.100 ± 0.001
	Fac.	0.504 ± 0.015	0.175 ± 0.029	0.067 ± 0.005
4×4	Bin.	4.37 ± 0.05	1.52 ± 0.03	0.315 ± 0.036
	Fac.	1.14 ± 0.02	0.283 ± 0.017	0.100 ± 0.003
6×6	Bin.	27.8 ± 2.6	6.98 ± 0.13	1.11 ± 0.04
	Fac.	4.22 ± 0.14	0.701 ± 0.031	0.244 ± 0.028
8×8	Bin.	131.8 ± 6.4	32.9 ± 1.2	4.20 ± 0.57
	Fac.	10.8 ± 0.6	1.40 ± 0.04	0.507 ± 0.033
12×12	Bin.	`TIMEOUT.`	161 ± 3.4	20.5 ± 1.3
	Fac.	68.9 ± 3.1	5.36 ± 0.12	1.82 ± 0.05
16×16	Bin.	`TIMEOUT.`	`TIMEOUT.`	94.2 ± 10.2
	Fac.	245 ± 2.3	14.8 ± 0.6	5.11 ± 0.46
32×32	Bin.	`TIMEOUT.`	`TIMEOUT.`	`TIMEOUT.`
	Fac.	`TIMEOUT.`	189 ± 10	58.9 ± 3.8

Los tiempos de la codificación binaria crecen más rápido que los de la factorizada a medida que aumenta el tamaño del grid. La Iteración de Valor domina t_{total} en la mayoría de las configuraciones.

5.5.2. Cociente de tiempos entre codificaciones

La Tabla 14 presenta el cociente $t^{\text{bin}}_{\text{total}}/t^{\text{fac}}_{\text{total}}$ para los pares donde ambas codificaciones completaron. `TIMEOUT` indica que la codificación binaria superó el límite de tiempo mientras que la factorizada no. Un valor menor que 1.0 indica que la binaria fue más rápida.

Tabla 14: Cociente de tiempos $t^{\text{bin}}_{\text{total}}/t^{\text{fac}}_{\text{total}}$ por configuración de inferencia.

Grid	**ddnnf**	**darwiche**	**sdd**
2×3	0.92×	1.06×	0.95×
3×3	1.60×	2.17×	1.49×
4×4	3.82×	5.36×	3.15×
6×6	6.59×	9.96×	4.56×
8×8	12.2×	23.6×	8.28×
12×12	`TIMEOUT`.	30.0×	11.2×
16×16	`TIMEOUT`.	`TIMEOUT`.	18.4×

Para el *grid* 2×3, las configuraciones *ddnnf* y *sdd* muestran cocientes menores a 1.0. A partir de 3×3, la codificación factorizada supera a la binaria en las tres configuraciones de inferencia..

La inversión de rendimiento entre 2×3 y 3×3 refleja el costo fijo de las Disyunciones Anotadas. Para el tamaño mínimo, el *overhead* de compilar los nodos *choice* supera la ventaja de un espacio de estados reducido; a partir de 3×3, la reducción de estados y cláusulas lo compensa.

5.5.3. Comparación de configuraciones de inferencia

Las tres configuraciones de inferencia difieren en su velocidad y en la tasa de crecimiento con el tamaño del *grid*. El análisis se realiza sobre la codificación factorizada, que ofrece el mayor número de configuraciones completadas.

La configuración *sdd* produce los tiempos más bajos en todos los tamaños evaluados. La configuración darwiche es entre 1.75× y 3.2× más lenta que *sdd*. La configuración *ddnnf* muestra la tasa de crecimiento más alta: en 4×4 es 4.0× más lenta que darwiche, y en 16×16 esa diferencia es 16.5×.

La diferencia entre *ddnnf* y *darwiche* confirma el análisis de la Sección 4.6. La ventaja de *sdd* frente a d-DNNF excede el alcance de este trabajo. Los tiempos se reportan solo como observación empírica. La Figura 8 presenta las seis series.

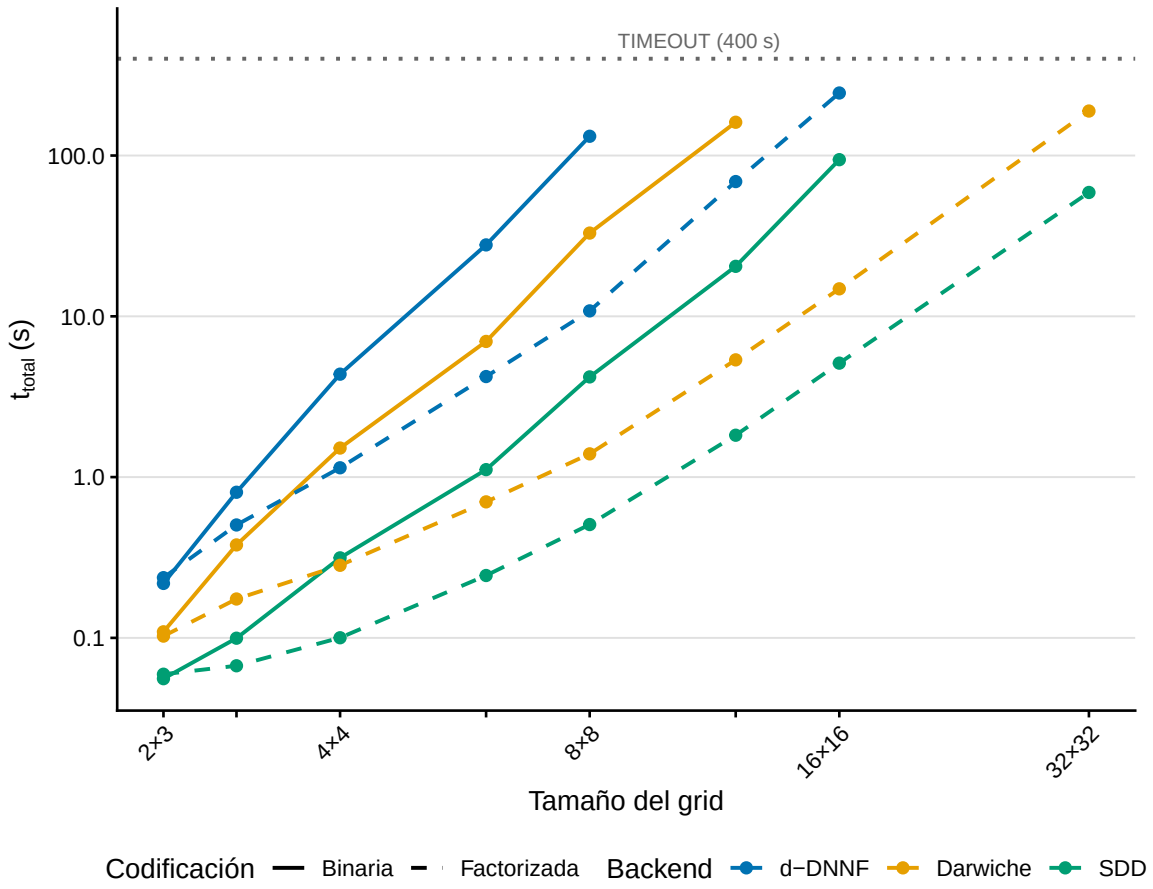

Figura 8: Tiempo total de ejecución en función del número de celdas del *grid* para las seis combinaciones codificación × *backend*.

5.6. Dominio Russell

El dominio de Mitchell utiliza transiciones deterministas, lo que aísla el efecto de la representación pero no verifica la extensión bajo condiciones estocásticas. El dominio canónico 3×4 de Russell y Norvig [23] complementa esta evaluación con transiciones probabilísticas, un estado trampa y un costo por paso. La evaluación se realiza con la codificación factorizada, que expresa la posición del agente como un fluente multivaluado `coor(X,Y)` con 11 valores posibles y un programa fuente de 50 reglas. Las configuraciones de inferencia evaluadas son *ddnnf* y darwiche; la configuración *sdd* produjo un error de compilación en este dominio y queda fuera del análisis cuantitativo.

5.6.1. Verificación de la función de valor óptima

La función de valor óptima $V^*(s)$ se verificó contra la Figura 17.3 de Russell y Norvig [23], con los parámetros canónicos del dominio ($\gamma = 1.0$, $R = -0.04$). La columna $|\Delta V^*|$ reporta la diferencia absoluta entre el valor obtenido por el solver y el valor de referencia.

Tabla 15: Verificación de $V^*(s)$ frente a la solución de referencia para el dominio Russell 3×4.

Fila	**Columna**	V^* **(referencia)**	$\lvert\Delta V^*\rvert$
3	1	0.812	< 0.001
3	2	0.868	< 0.001
3	3	0.918	< 0.001
2	1	0.762	< 0.001
2	3	0.660	< 0.001
1	1	0.705	< 0.001
1	2	0.655	< 0.001
1	3	0.611	< 0.001
1	4	0.388	< 0.001
3	4	meta $(+1)$	0.000
2	4	trampa (-1)	0.000

Las coordenadas (X, Y) siguen la convención del modelo: fila $X = 1$ en la base del *grid*, columna $Y = 1$ a la izquierda. El estado $(2, 2)$ es un obstáculo y no pertenece al espacio de estados.

Los nueve estados no terminales presentan diferencias menores a 0.001 respecto a los valores de referencia, y la política óptima π^* obtenida coincide con la Figura 17.2 de Russell y Norvig [23]. Los estados terminales tienen $V^* = 0$ en el solver, consistente con la semántica de absorción.

5.7. Discusión

La extensión preserva la semántica del MDP. Los resultados de las Secciones 5.3 y 5.6 confirman que la codificación multivaluada produce la misma V^* y la misma π^* que la codificación binaria tanto en condiciones deterministas como estocásticas. Esta compatibilidad es consecuencia de que ambas codificaciones representan el mismo MDP subyacente. La estructura de transiciones y recompensas es idéntica, de modo que la Iteración de Valor converge en el mismo número de pasos con los mismos parámetros [3].

Los datos de la Tabla 12 muestran que la codificación factorizada reduce el tamaño del programa fuente de manera consistente en todos los tamaños evaluados. El número de reglas se mantiene en 21 independientemente de la escala del dominio, mientras que la codificación binaria escala de 42 reglas en el grid 2×3 hasta 26 623 en el grid 32×32. Esta constancia es una consecuencia directa de que la codificación multivaluada describe la dinámica en términos de los valores naturales del dominio, sin necesidad de expandir cada variable en múltiples predicados booleanos. La reducción del número de reglas va acompañada de la eliminación de los estados espurios, lo que reduce el número de pares (s, a) que el *solver* evalúa en cada iteración de Bellman.

La codificación multivaluada y el evaluador de Darwiche producen efectos que se amplifican mutuamente. La codificación factorizada reduce $|\mathcal{S}|$ y, con ello, el número de pares (s, a) por iteración. El evaluador de Darwiche [20] elimina el factor Q del costo de cada par al calcular todas las marginales en dos pasadas lineales sobre el circuito. Ambos efectos escalan con el tamaño del *grid*, lo que explica el crecimiento del cociente en la Tabla 14. El grid 2×3 constituye un caso límite donde la diferencia entre espacios de estados (8 frente a 6 estados) no compensa el costo fijo de compilar los nodos *choice* que implementan las Disyunciones Anotadas; a partir del grid 3×3, la reducción del espacio de estados domina el *overhead* de compilación y la codificación factorizada supera a la binaria en las tres configuraciones de inferencia evaluadas.

Los resultados son válidos dentro del alcance evaluado. Los dominios Mitchell son *grids* regulares sin obstáculos, con transiciones deterministas y un único fluente; la generalización a dominios con múltiples fluentes dependientes, transiciones complejas u obstáculos requiere evaluación adicional. El dominio Russell aporta verificación estocástica en una única instancia de 11 estados, insuficiente para derivar tendencias de escalamiento. El comportamiento de la configuración *sdd* en dominios estocásticos queda como cuestión abierta; el error de compilación observado en el dominio Russell no fue diagnosticado en este trabajo.

6. Conclusiones

6.1. Síntesis del trabajo realizado

MDP-ProbLog se extendió para representar fluentes de estado multivaluados mediante Disyunciones Anotadas (ADs), un constructo nativo de la Programación Lógica Probabilística que garantiza la exclusión mutua entre los posibles valores de una variable por construcción semántica [11]. Bajo esta representación, el espacio de estados adopta una estructura en base mixta donde cada fluente opera con su cardinalidad real, lo que elimina los estados espurios generados por la binarización forzada de la versión original y traslada la responsabilidad de la exclusión mutua del modelador al motor lógico.

La intervención se limita a la capa de representación que alimenta al compilador de ProbLog, sin alterar los algoritmos de inferencia ni los circuitos compilados. Ello preserva la compatibilidad con todos los modelos booleanos existentes y las garantías formales de Bueno et al. [4].

La evaluación empírica sobre los dominios de Mitchell [9] y Russell y Norvig [23] confirmó que ambas codificaciones producen la misma V^* y la misma π^* para un mismo MDP, en condiciones deterministas y estocásticas. Los experimentos mostraron además que la codificación multivaluada mantiene el tamaño del programa fuente constante respecto al tamaño del dominio y elimina los estados espurios. La mejora computacional resultante crece con la escala del problema.

6.2. Contribuciones

Se aporta un esquema factorizado del espacio de estados bajo una estructura *mixed-radix* que asigna a cada fluente su cardinalidad real. A diferencia de la representación booleana original, donde el tamaño del espacio de estados crece como potencia de dos del número de bits independientemente de la semántica del dominio, la representación *mixed-radix* garantiza que cada índice corresponda exactamente a un estado válido. Esta propiedad elimina por construcción los estados espurios que la codificación booleana introduce cuando el número de estados del dominio no es una potencia exacta de dos [6]. Junto con el esquema factorizado, se desarrolló un clasificador automático de fluentes que determina, a partir de un índice invertido sobre los nodos `choice` de la base de cláusulas, si un fluente declarado por el usuario es booleano o multivaluado. El clasificador soporta tanto la inferencia implícita del tipo, mediante el análisis de las ADs presentes en el programa, como la declaración explícita mediante el predicado `state_fluent/2`.

La integración opera en la fase de inyección de hechos, donde sustituye los hechos probabilísticos independientes por ADs uniformes para los fluentes multivaluados. Ello preserva las garantías de inferencia exacta sobre circuitos d-DNNF y SDD establecidas por Darwiche y Marquis [18, 22]. El sistema conserva además la retrocompatibilidad con los modelos booleanos existentes. Cuando el programa del usuario no contiene ADs, el *pipeline* se comporta de forma idéntica al de Bueno et al. [4].

Se implementó también el evaluador de diferenciación de circuitos de Darwiche [20], que calcula todas las marginales requeridas durante la Iteración de Valor en dos pasadas lineales sobre el circuito compilado. Este evaluador reduce el costo de evaluar Q consultas simultáneas de $\mathcal{O}(Q \times |C|)$ a $\mathcal{O}(|C|) + \mathcal{O}(Q)$, donde $|C|$ denota el tamaño del circuito. El beneficio es multiplicativo con el de la codificación factorizada: aquella reduce el número de pares estado-acción por iteración, mientras que este reduce el costo de procesar cada par. El sistema admite además modelos mixtos en los que fluentes booleanos y multivaluados coexisten en el mismo programa, lo que permite una adopción gradual sin reescribir los modelos existentes.

6.3. Limitaciones técnicas identificadas

Durante el desarrollo e implementación se identificaron limitaciones técnicas distintas de las restricciones de diseño anticipadas en la Sección 1.5.2.

1. **Agrupamiento global de fluentes.** El clasificador agrupa bajo un único fluente multivaluado todos los términos que comparten predicado y aridad. Esta convención impide representar dominios en los que múltiples entidades independientes comparten el mismo predicado con valores distintos.

2. **Incompatibilidad entre la configuración SDD y dominios estocásticos.** La configuración de inferencia *sdd* generó un error de compilación en el dominio Russell, que presenta transiciones estocásticas no triviales. La causa no fue diagnosticada; se presume una restricción del *backend* SDD de ProbLog2 ante ciertos patrones de ADs.

3. **Supuesto de independencia condicional no verificado en tiempo de ejecución.** El cálculo del valor esperado factorizado asume que las distribuciones de transición de cada fluente son condicionalmente independientes dado el estado y la acción actuales. El sistema no detecta ni advierte si las reglas del usuario introducen dependencias entre fluentes que violan este supuesto, lo que puede producir resultados incorrectos sin aviso.

6.4. Trabajo futuro

Las limitaciones identificadas y la estructura de la extensión propuesta sugieren las siguientes líneas de desarrollo e investigación.

1. **Fluentes con múltiples entidades.** La limitación de agrupamiento global impide modelar con naturalidad dominios en los que varias entidades independientes comparten el mismo predicado de fluente. Extender el clasificador para operar por entidad, diferenciando instancias a partir de sus argumentos de identificación, cubriría una clase más amplia de dominios sin requerir declaraciones adicionales del usuario.

2. **Resolución simbólica del espacio de estados.** Esta extensión no modificó la estrategia de resolución enumerativa de MDP-ProbLog; el algoritmo de Iteración de Valor continúa recorriendo todos los pares estado-acción en cada iteración. Vale la pena explorar la viabilidad de adaptar ideas historicamente aplicadas a otro tipo de representaciones simbólicas, como los Diagramas de Decisión Algebraicos de SPUDD [6] y extraporalas a los Diagramas de Decisión Sentenciales o los d-DNNFs. Esto permitiría manipular la función de valor sin enumerar el espacio de estados y superaría los límites de escalamiento observados en los experimentos.

3. **Impacto de las Disyunciones Anotadas en el tamaño del circuito compilado.** Los experimentos mostraron que la codificación factorizada produce tiempos de resolución menores, pero el mecanismo por el cual las ADs afectan el tamaño del circuito d-DNNF o SDD no fue modelado teóricamente. Cuantificar esta relación permitiría predecir el comportamiento del compilador ante distintas estructuras de fluentes multivaluados y orientar decisiones de diseño de modelos.

Índice de figuras

1. Representación por coordenadas vs. codificación booleana con estados inalcanzables para el grid 2×3. 3
2. Grid de navegación 2×3 con transiciones estocásticas. 14
3. Política óptima del grid $2{\times}3$. (a) Acción óptima $\pi^*(r, c)$; (b) valor $V^*(r, c)$ calculado por Iteración de Valor con $\gamma = 0.9$, $\varepsilon = 0.1$. 22
4. Diagrama de la arquitectura original del framework MDP-ProbLog. 26
5. pipeline de clasificación. Se explica la ejecución del método `classify()` . . . 33
6. pipeline de preparación. Se explica la ejecución del método `__prepare()` . . . 35
7. Número de reglas del programa fuente en función del número de celdas del *grid*. 49
8. Tiempo total de ejecución en función del número de celdas del *grid* para las seis combinaciones codificación $\times$ *backend*. 52

Índice de cuadros

1. Codificación binaria de las 6 celdas de un grid de 2×3. 3
2. Escalamiento de estados espurios en la codificación binaria por tamaño de grid. 4
3. Codificación binaria forzada para una variable ternaria en el dominio *Factory*. 5
4. Comparativa de los enfoques del estado del arte. 12
5. Mundos posibles inducidos por la semántica de distribución sobre el Código 1. 17
6. Distribución generada por la AD del Código 3. Cada selección corresponde a un único destino posible. 19
7. Codificación *mixed-radix* para el dominio color $\times$ velocidad. 30
8. Comparación de costo asintótico entre los evaluadores de ProbLog. 42
9. Métodos de exportación del `CSVExporter` y estructuras producidas. 43
10. Parámetros del método `Simulator.run()`. 44
11. Verificación cruzada de corrección funcional en el dominio Mitchell. 47
12. Métricas del programa fuente por codificación y tamaño de *grid*. 48
13. Tiempo total de ejecución t_{total} (segundos) por codificación y configuración de inferencia. 50
14. Cociente de tiempos $t^{\text{bin}}_{\text{total}}/t^{\text{fac}}_{\text{total}}$ por configuración de inferencia. 51
15. Verificación de $V^*(s)$ frente a la solución de referencia para el dominio Russell 3×4. 53

Índice de algoritmos

1. Inferencia de tipo por intersección de orígenes estocásticos 33
2. Valor esperado del estado sucesor sobre el esquema factorizado 38
3. Diferenciación de circuitos d-DNNF (Darwiche, 2003) 41

Referencias

[1] Luc De Raedt, Angelika Kimmig y Hannu Toivonen. «ProbLog: A probabilistic prolog and its application in link discovery». En: *IJCAI*. Vol. 7. 2007, págs. 2462-2467.

[2] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.

[3] Martin L Puterman. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.

[4] Thiago P Bueno et al. «Markov decision processes specified by probabilistic logic programming: Representation and solution». En: *2016 5th Brazilian Conference on Intelligent Systems (BRACIS)*. IEEE. 2016, págs. 337-342.

[5] Craig Boutilier, Thomas Dean y Steve Hanks. «Decision-theoretic planning: Structural assumptions and computational leverage». En: *Journal of Artificial Intelligence Research* 11 (1999), págs. 1-94.

[6] Jesse Hoey et al. «SPUDD: Stochastic Planning using Decision Diagrams». En: *Proceedings of the 15th Conference on Uncertainty in Artificial Intelligence (UAI-99)*. 1999, págs. 279-288.

[7] Davide Nitti, Vaishak Belle y Luc De Raedt. «Planning in discrete and continuous Markov decision processes by probabilistic programming». En: *Machine Learning* 104.2-3 (2016), págs. 327-342.

[8] Héctor Avilés et al. «Autonomous Behavior Selection For Self-driving Cars Using Probabilistic Logic Factored Markov Decision Processes». En: *Applied Artificial Intelligence* 38.1 (2024), pág. 2304942.

[9] Tom M. Mitchell. *Machine Learning*. New York: McGraw-Hill, 1997.

[10] Richard S Sutton y Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.

[11] Joost Vennekens, Sofie Verbaeten y Maurice Bruynooghe. «Logic programs with annotated disjunctions». En: *International Conference on Logic Programming*. Springer, 2004, págs. 431-445.

[12] Craig Boutilier, Richard Dearden y Moisés Goldszmidt. «Stochastic dynamic programming with factored representations». En: *Artificial Intelligence* 121.1-2 (2000), págs. 49-107.

[13] Scott Sanner. «Relational Dynamic Influence Diagram Language (RDDL): Language Description». Unpublished Manuscript, Australian National University. 2010.

[14] Luc De Raedt y Angelika Kimmig. «Probabilistic (logic) programming concepts». En: *Machine Learning* 100.1 (2015), págs. 5-47.

[15] Guy Van den Broeck et al. «DTProbLog: A Decision-Theoretic Probabilistic Prolog». En: *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence (AAAI-10)*. AAAI Press, 2010, págs. 1217-1222.

[16] Joost Vennekens, Marc Denecker y Maurice Bruynooghe. «CP-logic: A language of causal probabilistic events and its relation to logic programming». En: *Theory and Practice of Logic Programming* 9.3 (2009), págs. 245-308.

[17] David Poole. «The Independent Choice Logic for Modelling Multiple Agents under Uncertainty». En: *Artificial Intelligence* 94.1-2 (1997), págs. 7-56.

[18] Adnan Darwiche y Pierre Marquis. «A knowledge compilation map». En: *Journal of Artificial Intelligence Research* 17 (2002), págs. 229-264.

[19] Adnan Darwiche. «SDD: A New Canonical Representation of Propositional Knowledge Bases». En: *Proceedings of the 22nd International Joint Conference on Artificial Intelligence (IJCAI)*. 2011, págs. 819-826.

[20] Adnan Darwiche. «A differential approach to inference in Bayesian networks». En: *Journal of the ACM* 50.3 (2003), págs. 280-305.

[21] Anton Dries et al. «ProbLog2: Probabilistic Logic Programming». En: *Machine Learning and Knowledge Discovery in Databases*. Ed. por Albert Bifet et al. Cham: Springer International Publishing, 2015, págs. 312-315.

[22] Daan Fierens et al. «Inference and learning in probabilistic logic programs using weighted Boolean formulas». En: *Theory and Practice of Logic Programming* 15.3 (2015), págs. 358-401.

[23] Stuart Jonathan Russell y Peter Norvig. *Artificial Intelligence: A Modern Approach*. English. Third. England: Pearson Education, 2016. ISBN: 9781292153964.

[24] Taisuke Sato. «A statistical learning method for logic programs with distribution semantics». En: *Proceedings of the 12th International Conference on Logic Programming (ICLP'95)*. Citeseer, 1995.